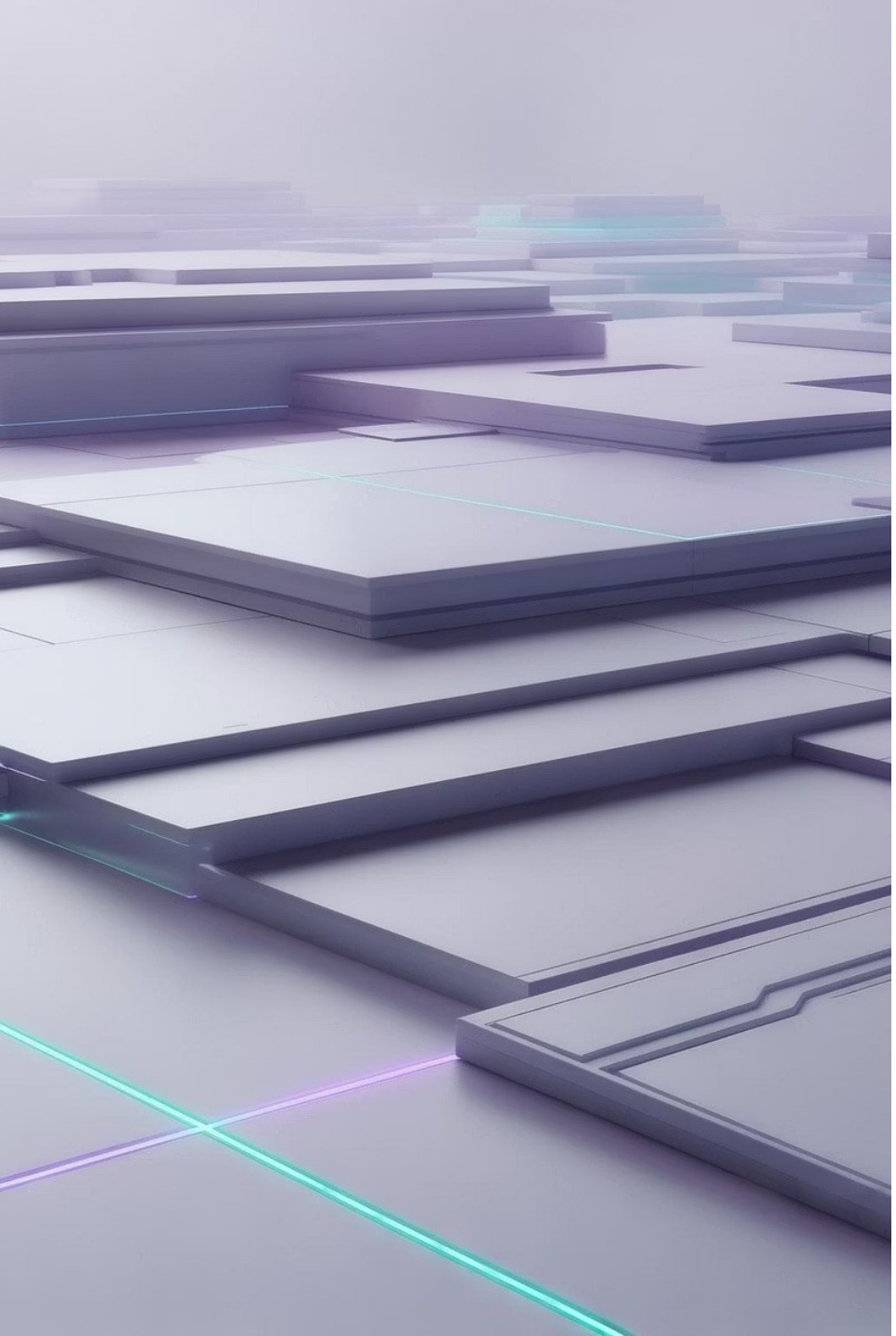


WEB-ENGINEERING 1 · TERMIN 4 / 9

LAYOUT & TAILWIND

Flexbox, Grid, Responsive – und warum Tailwind

TAISC The AI Software Company

Joshua Heller · 20.10.2026

RÜCKBLICK

Wer zeigt seine Hausaufgabe?

Zwei Freiwillige, je vier Minuten – dafür gibt es einen Bonuspunkt.

Freiwillig

Niemand muss. Wer zeigt, bekommt +1 Punkt fürs Team (max. +5 im Semester).

Halb fertig reicht

Keine Musterschüler nötig – gerade an Baustellen lernen alle am meisten.

Hängen geblieben?

Auch das zählt: Zeigt, wo es nicht weiterging – wir lösen es gemeinsam.

Letzte Woche

Eigenes Stylesheet: Farben, Schrift, Navigation mit Hover.

© 2026 TAISC – The AI Software Company GmbH

KURZ ZURÜCK

Das war letzte Woche, das kommt heute

Letzte Woche

Das solltet ihr mitgenommen haben

- Selektoren und Spezifität: wer gewinnt
- Das Box-Modell und border-box
- position: sticky, absolute, z-index
- Moderne Features: :has, clamp, Dark Mode

Heute

Darauf bauen wir auf

- Flexbox und Grid: Elemente nebeneinander
- Mobile first und Breakpoints
- Tailwind: CSS mit kurzen Namen
- npm, package.json und Vite

© 2026 TAISC – The AI Software Company GmbH

WO WIR STEHEN

Euer Projekt wächst mit jedem Termin

Heute kommt dazu: Layout + Tailwind

06.10.
HTML-Gerüst
Seiten, Navigation, Formular

13.10.
Eigenes CSS
Farben, Schrift, Abstände

20.10.
Layout + Tailwind
Flex, Grid, responsive

27.10.
Deployed
Live-URL auf GitHub Pages

17.11.
Interaktiv
JavaScript, Events, Zustand

24.11.
Echte Daten
fetch aus einer öffentlichen API

01.12.
Feinschliff
Komponenten, Bonus, README

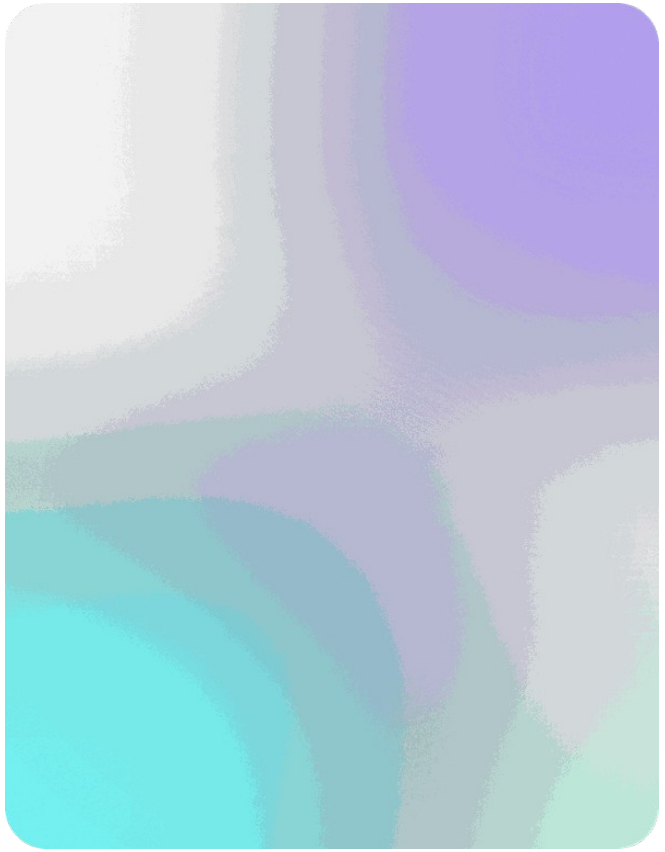
08.12.
Präsentation
Demo + Rückfragen

© 2026 TAISC – The AI Software Company GmbH

KAPITEL 1

Layout

Wie Elemente nebeneinander kommen.



KURZE GESCHICHTE

Dreißig Jahre Layout-Hacks

1990er: Tabellen

<table> für alles

- Ganze Seiten als Tabelle
- Schwer zu ändern, nicht barrierefrei

2000er: float

eigentlich für Bilder im Text

- Spalten mit float: left
- Clearfix-Hacks gegen Chaos
- Steht noch in alten Tutorials – und in KI-Antworten

Heute: Flexbox & Grid

endlich für Layout gebaut

- Flexbox seit ~2015 überall
- Grid seit ~2017 überall
- Wenn die KI float fürs Layout nimmt: nachfragen

© 2026 TAISC – The AI Software Company GmbH

NORMALER FLUSS

Bevor ihr zu Flex und Grid greift

Viele Layouts brauchen gar nichts davon. Der Browser ordnet schon von selbst an.

Block-Elemente

div, p, section, h1 … stehen untereinander und nehmen die volle Breite.

Inline-Elemente

a, span, strong … fließen im Text nebeneinander und brechen um.

Der Fluss

Reihenfolge im HTML = Reihenfolge auf dem Bildschirm. Gut für Screenreader.

Die Frage

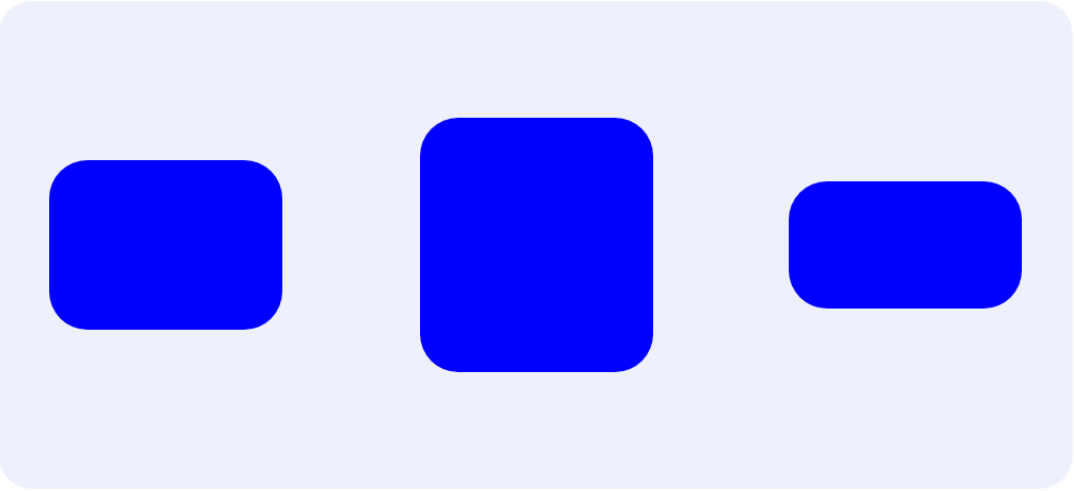
Was soll sich nebeneinander anordnen, und in wie vielen Richtungen? Daraus folgt Flex oder Grid.

© 2026 TAISC – The AI Software Company GmbH

FLEXBOX & GRID

Eine Richtung oder zwei

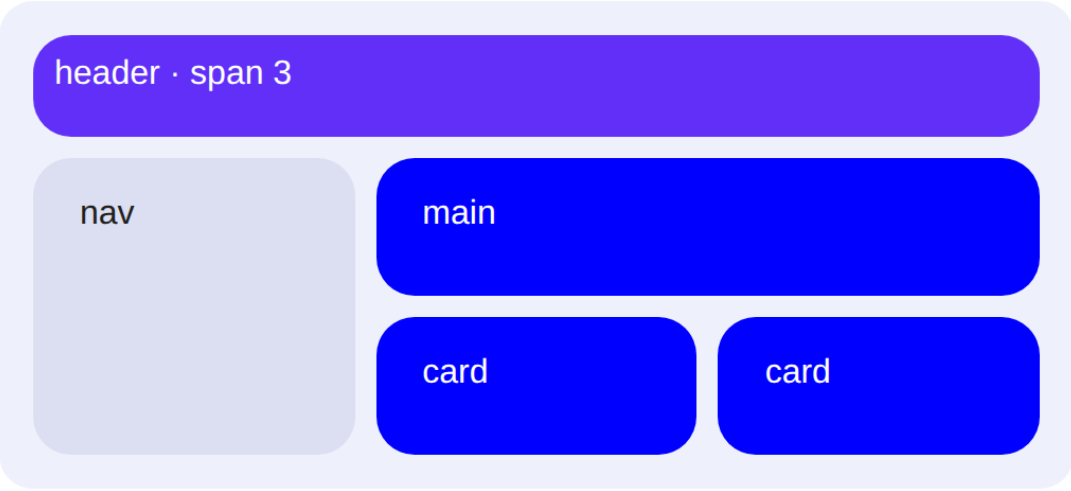
DISPLAY: FLEX



⟶ **Hauptachse** (row): justify-content

↕ **Querachse**: align-items

DISPLAY: GRID



Flex = eine Richtung. **Grid** = Zeilen und Spalten gleichzeitig.

© 2026 TAISC – The AI Software Company GmbH

DISPLAY

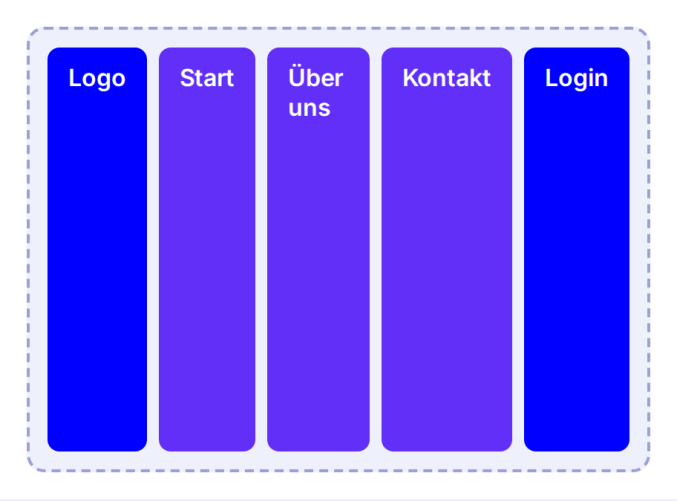
Eine Zeile schaltet das Layout um

display: block;



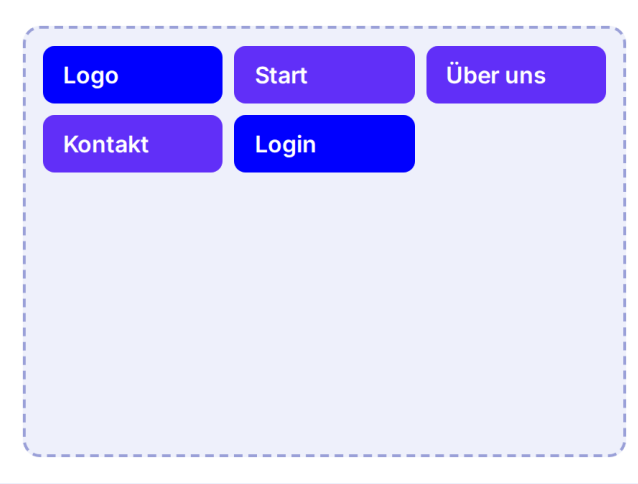
Standard: untereinander, volle Breite

display: flex;



Eine Reihe, jedes Element so breit wie sein Inhalt

display: grid;
grid-template-columns: repeat(3, 1fr);



Raster mit drei gleich breiten Spalten

display steht immer am Eltern-Element (dem Container). Seine direkten Kinder ordnen sich dann an.

© 2026 TAISC – The AI Software Company GmbH

FLEXBOX

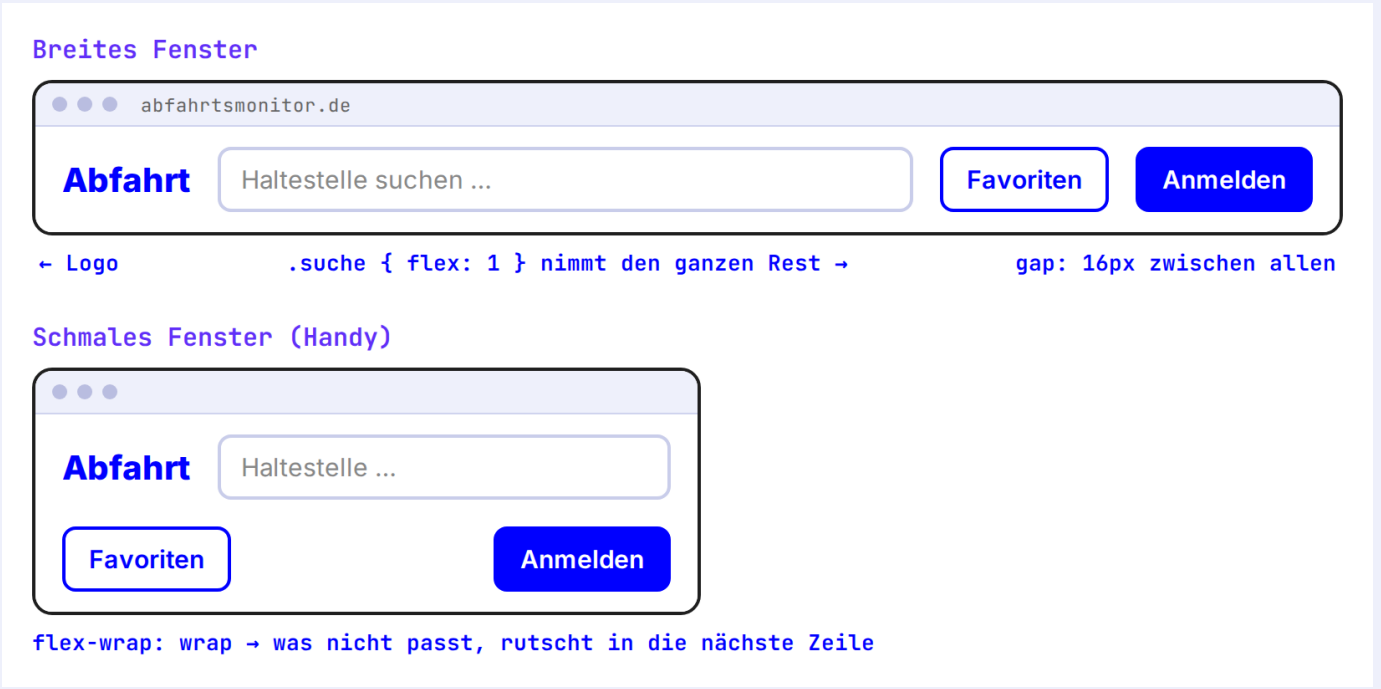
Fünf Eigenschaften für 90 % der Fälle

style.css

```
nav {
  display: flex;
  flex-direction: row;
  justify-content: space-between;
  align-items: center;
  gap: 1rem;
  flex-wrap: wrap;
}

.suche {
  flex: 1;   /* nimmt den Rest */
}
```

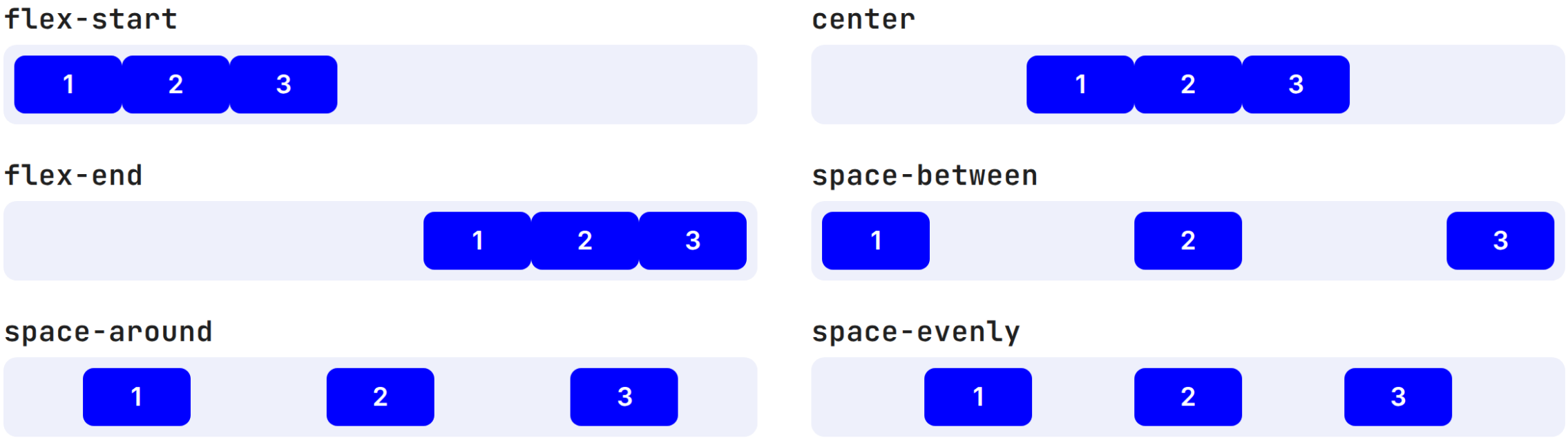
Was daraus wird



© 2026 TAISC – The AI Software Company GmbH

FLEXBOX IN BILDERN

justify-content in sechs Varianten

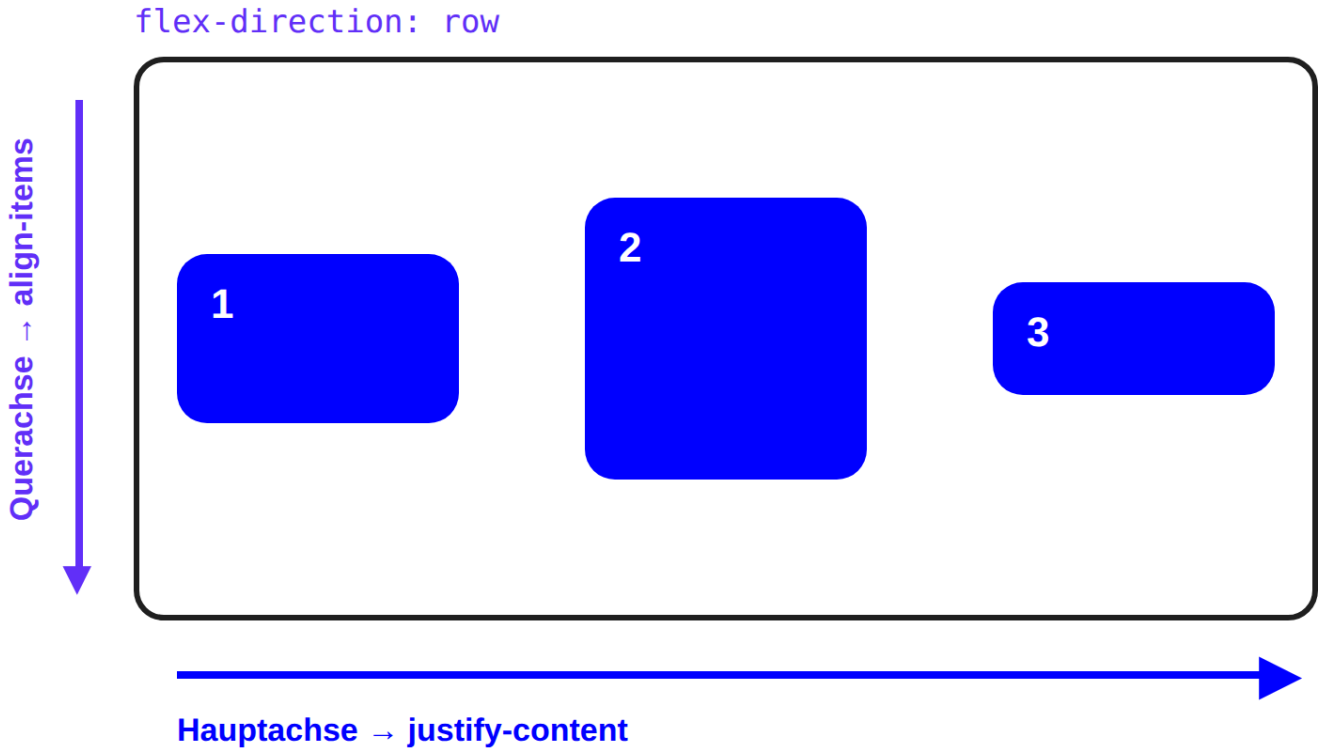


Drei Elemente, sechs Werte. Live durchklicken: Layout-Spielwiese, Reiter „Flex oder Grid“.

© 2026 TAISC – The AI Software Company GmbH

FLEXBOX-ACHSEN

Zwei Achsen, zwei Eigenschaften



`justify-content`
Verteilung entlang der Hauptachse: flex-start, center, space-between, space-around, space-evenly

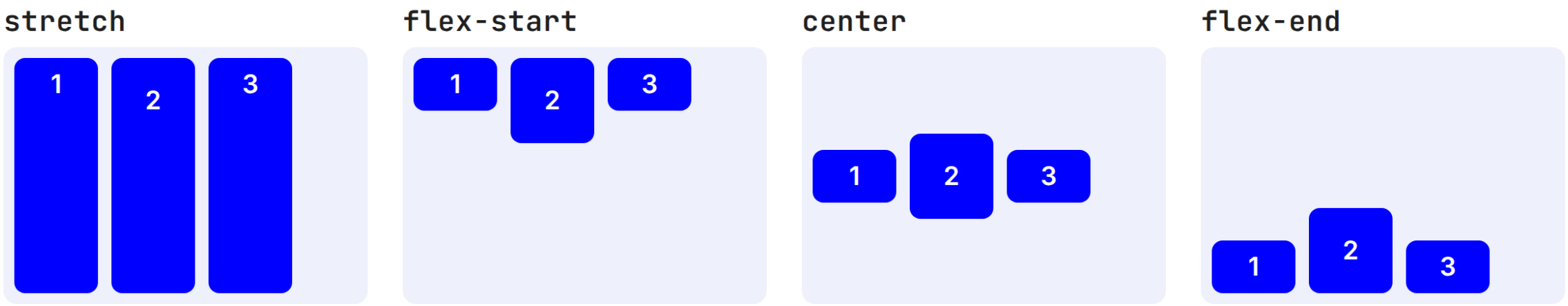
`align-items`
Ausrichtung quer dazu: stretch (Standard), center, flex-start, flex-end, baseline

Bei column drehen sich die Achsen
justify wirkt dann vertikal, align horizontal.

© 2026 TAISC – The AI Software Company GmbH

FLEXBOX IN BILDERN

align-items in vier Varianten



stretch ist der Standard: alle werden so hoch wie das höchste Element.

Element 2 ist höher als die anderen. align-items entscheidet, wo die kleineren stehen.

© 2026 TAISC – The AI Software Company GmbH

FLEX: 1 ENTSCHLÜSSELT

Wer bekommt den freien Platz?

```
/* nichts gesetzt */
```




```
.suche { flex: 1; }
```




```
* { flex: 1; } /* alle */
```




```
.suche { flex: 2; }
.rest  { flex: 1; }
```




```
.logo { flex: 0 0 240px; }
```



flex: 1 ist die Kurzform für flex: 1 1 0 – darf wachsen, darf schrumpfen, startet bei 0 und teilt sich den Rest.

© 2026 TAISC – The AI Software Company GmbH

MIN-WIDTH: 0

Wenn Text die Flex-Box sprengt

Gleiche Karte, 400 px breit

Ohne min-width: 0



Mit min-width: 0 + overflow + ellipsis



style.css

```
.karte { display: flex; }
.text {
  flex: 1;
  min-width: 0;    /* darf schrumpfen */
}
.text h4 {
  white-space: nowrap;      /* 1 Zeile */
  overflow: hidden;         /* abschneiden */
  text-overflow: ellipsis;  /* „…“ */
}
```

Flex-Kinder haben standardmäßig min-width: auto – sie werden nie schmaler als ihr Inhalt. min-width: 0 hebt das auf.

© 2026 TAISC – The AI Software Company GmbH

KENNT IHR DAS?

Zentrieren, früher

Suchverlauf · Erstes Semester, Woche 4

09:12	wie zentriere ich ein div
09:14	wie zentriere ich ein div vertikal
09:31	wie zentriere ich ein div vertikal UND horizontal
09:47	text-align center geht nicht warum
10:05	margin: 0 auto geht auch nicht
10:22	css zentrieren 2026 funktioniert wirklich
10:40	ist es normal css zu hassen
10:41	**display: grid; place-items: center;**

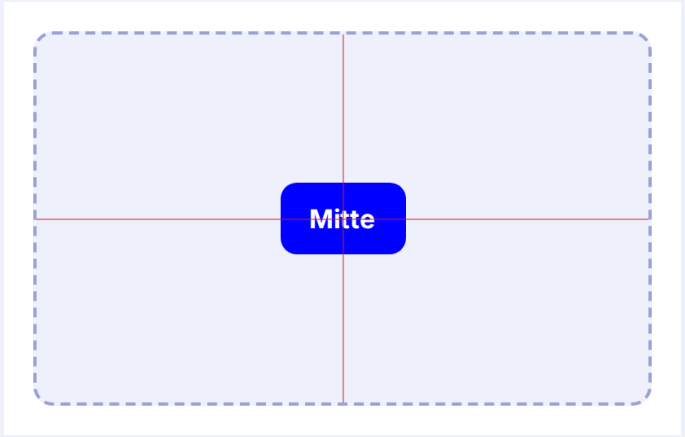
Jahrzehntelang der Running Gag der Webentwicklung. Heute ist es eine Zeile.

© 2026 TAISC – The AI Software Company GmbH

ZENTRIEREN

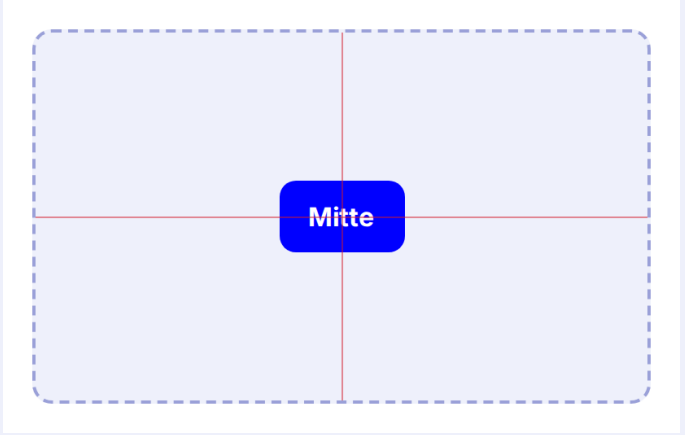
Drei Wege, ein Ergebnis

display: flex;
justify-content: center;
align-items: center;



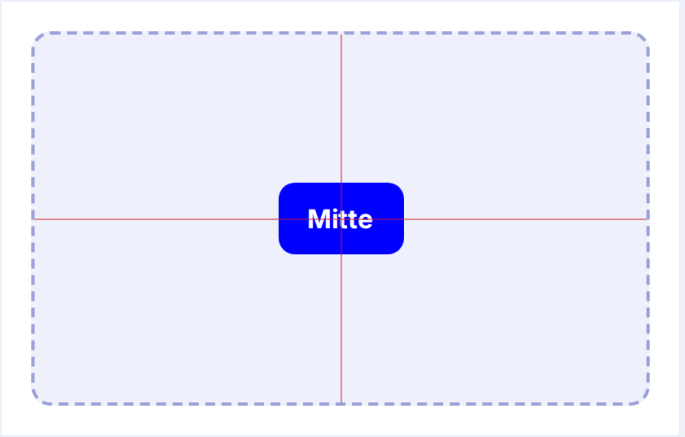
Klassiker: beide Achsen einzeln

display: grid;
place-items: center;



Kürzeste Form

display: flex;
/* am Kind: */
margin: auto;



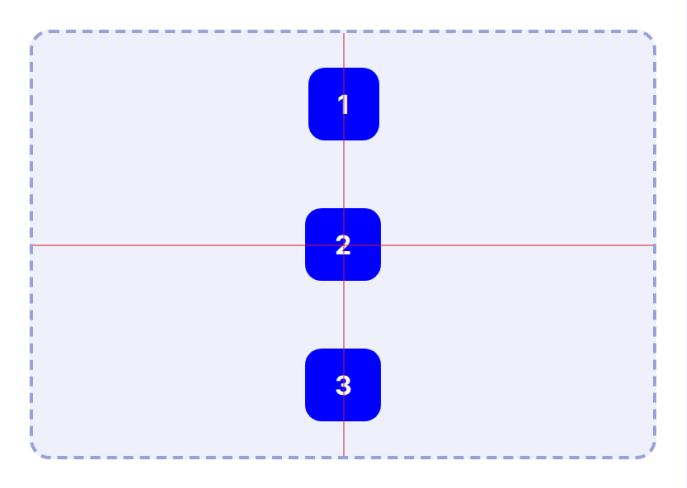
Das Kind schiebt sich selbst in die Mitte

© 2026 TAISC – The AI Software Company GmbH

FEINHEIT

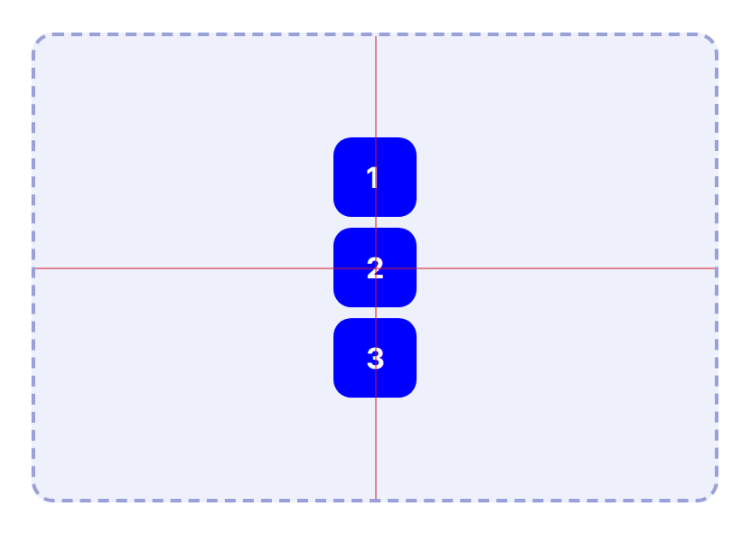
place-items oder place-content?

display: grid;
place-items: center;



Jedes Kind wird in seiner eigenen Zeile zentriert – die Zeilen füllen die ganze Höhe

display: grid;
place-content: center;



Der ganze Block wird als Paket in die Mitte geschoben

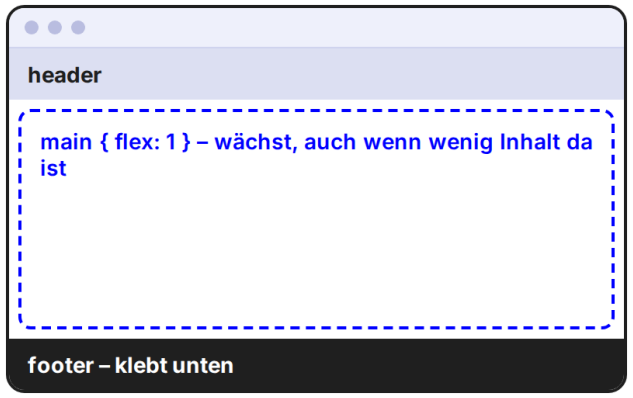
Mit einem Kind sehen beide gleich aus. Den Unterschied seht ihr erst bei mehreren Kindern.

© 2026 TAISC – The AI Software Company GmbH

FLEXBOX-MUSTER 1/2

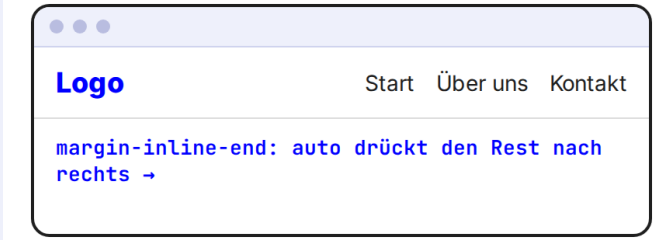
Layouts, die ihr ständig braucht

```
body { display: flex;
  flex-direction: column;
  min-height: 100dvh; }
main { flex: 1; }
```



Footer immer unten

```
nav { display: flex;
  align-items: center; }
.logo {
  margin-inline-end: auto; }
```



Logo links, Links rechts

```
.media { display: flex;
  gap: 1rem;
  align-items: flex-start; }
.media img { flex: none; }
```

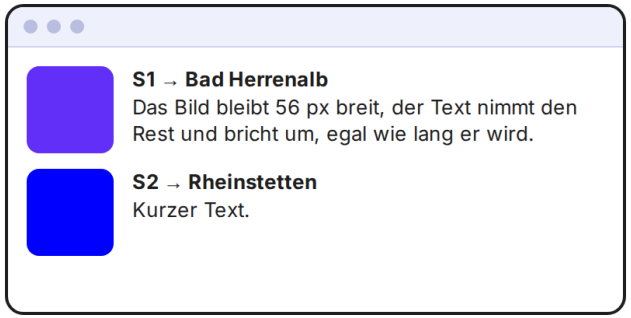


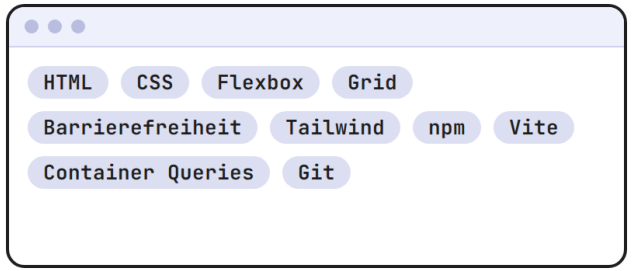
Bild neben Text

© 2026 TAISC – The AI Software Company GmbH

FLEXBOX-MUSTER 2/2

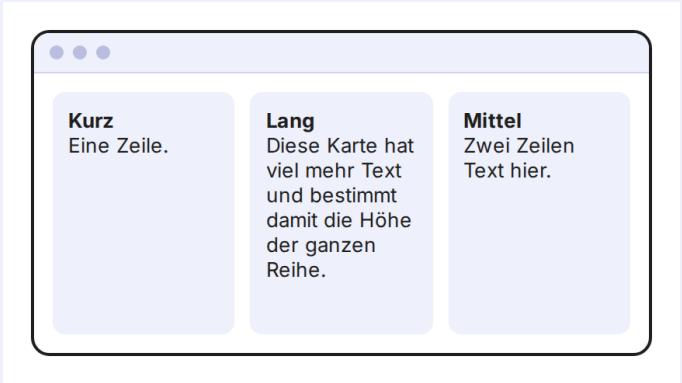
Noch zwei Muster – und was 100dvh heißt

```
.tags { display: flex;
  flex-wrap: wrap;
  gap: .5rem; }
```



Tags, die umbrechen

```
.reihe { display: flex; }
/* align-items: stretch
   ist Standard */
```



Karten automatisch gleich hoch

100vh oder 100dvh?

- vh: 1 % der Fensterhöhe – auf dem Handy die größte mögliche, also mit eingeklappter Adressleiste
- dvh (dynamic viewport height): die gerade sichtbare Höhe
- min-height: 100dvh → Seite ist mindestens bildschirmhoch, nichts rutscht hinter die Browserleiste

© 2026 TAISC – The AI Software Company GmbH

GRID: FR

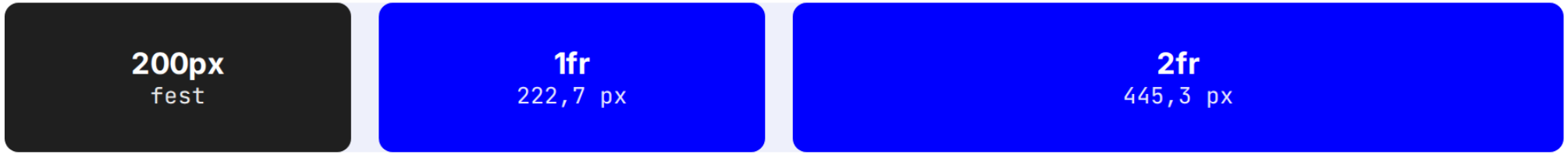
fr = ein Anteil am freien Platz

`grid-template-columns: 1fr 1fr 1fr; gap: 16px; Breite 900 px`



900 - 2 × 16 (gap) = 868 px frei → 868 ÷ 3 Anteile = 289,3 px pro fr

`grid-template-columns: 200px 1fr 2fr;`



900 - 200 (fest) - 32 (gaps) = 668 px frei → 668 ÷ 3 Anteile = 222,7 px pro fr → 2fr = 445,3 px

fr ist keine feste Größe wie px oder %. Erst wird abgezogen, was fest ist (px und gap), der Rest wird nach Anteilen verteilt.

© 2026 TAISC – The AI Software Company GmbH

AUTO-FIT + MINMAX

Responsive Karten ohne Media Query

style.css

```
.karten {
  display: grid;
  grid-template-columns:
    repeat(auto-fit,
      minmax(220px, 1fr));
  gap: 1rem;
}

/* Lies: So viele Spalten,
   wie mit mindestens
   220 px hineinpassen.
   Jede bekommt 1fr
   vom Rest. */
```

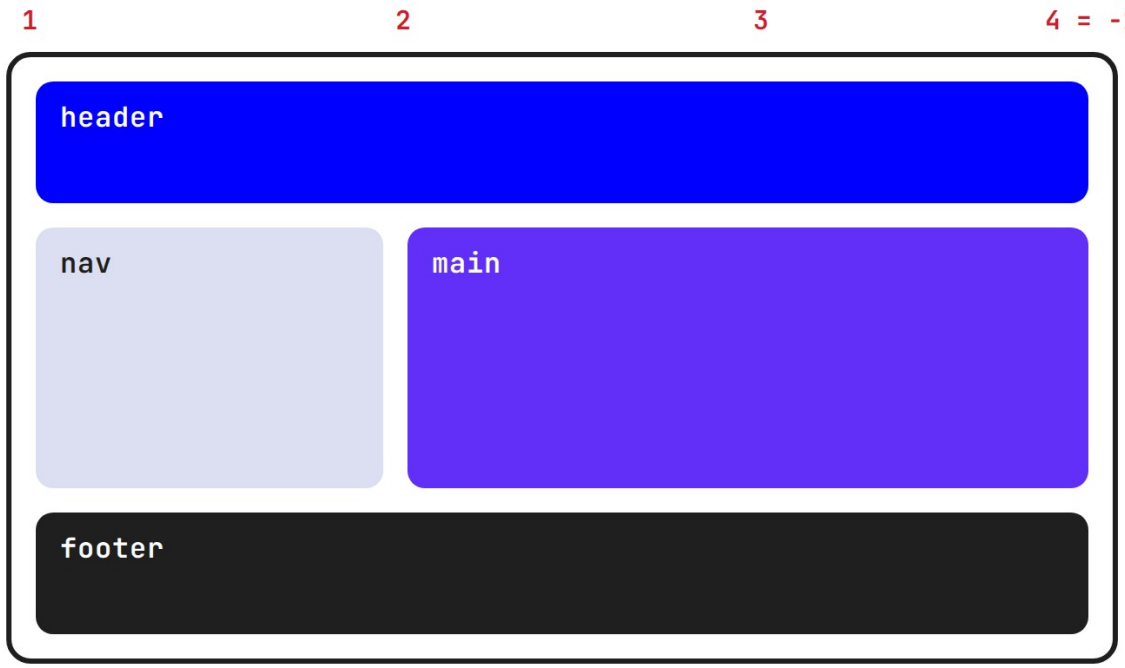
Dieselbe Regel bei drei Fensterbreiten



© 2026 TAISC – The AI Software Company GmbH

GRID-VOKABULAR

Linien, Spuren, Bereiche im Code




```
.seite {
  display: grid;
  grid-template-columns: 200px 1fr 1fr;  /* 3 Spuren */
  gap: 14px;                             /* Abstand */
}
header { grid-column: 1 / -1; }  /* Linie 1 bis letzte */
nav    { }                       /* nimmt nächste Zelle */
main   { grid-column: span 2; } /* 2 Spuren breit */
footer { grid-column: 1 / -1; }
```

Linien zählen ab 1, von hinten ab -1. span 2 heißt: zwei Spuren breit, egal wo das Element anfängt.

© 2026 TAISC – The AI Software Company GmbH

GRID-AREAS

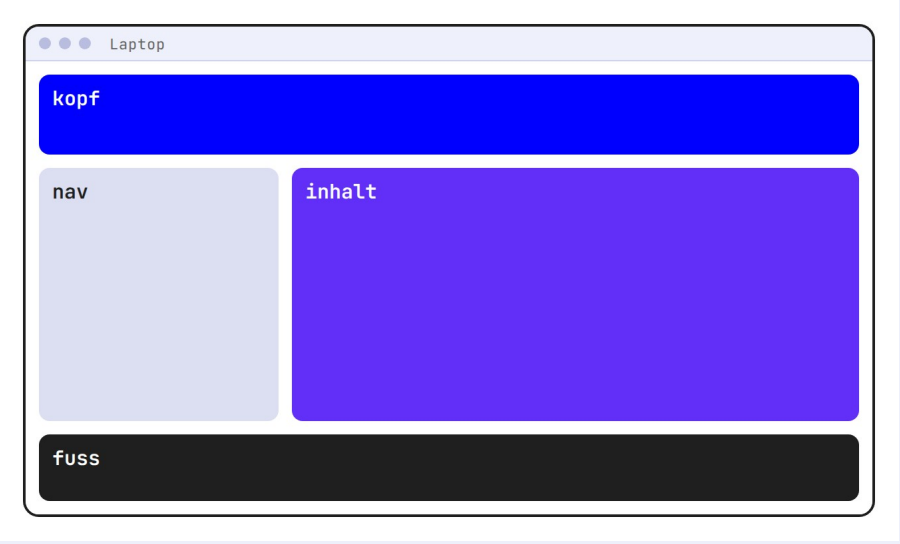
Das Layout als Skizze im Code

style.css

```
.seite {
  display: grid;
  grid-template-columns: 180px 1fr;
  grid-template-areas:
    "kopf kopf"
    "nav  inhalt"
    "fuss fuss";
}
header { grid-area: kopf; }
nav    { grid-area: nav; }
main   { grid-area: inhalt; }
footer { grid-area: fuss; }

@media (width < 640px) {
  .seite {
    grid-template-columns: 1fr;
    grid-template-areas:
      "kopf" "nav" "inhalt" "fuss";
  }
}
```

Laptop



Jeder String ist eine Zeile im Raster

Handy



Neue Skizze, gleiches HTML

© 2026 TAISC – The AI Software Company GmbH

AUTO-FIT VS. AUTO-FILL

auto-fit oder auto-fill?

repeat(auto-fit, minmax(130px, 1fr))



Leere Spalten fallen weg – die Karten dehnen sich

repeat(auto-fill, minmax(130px, 1fr))



Leere Spalten bleiben stehen – die Karten behalten ihre Breite

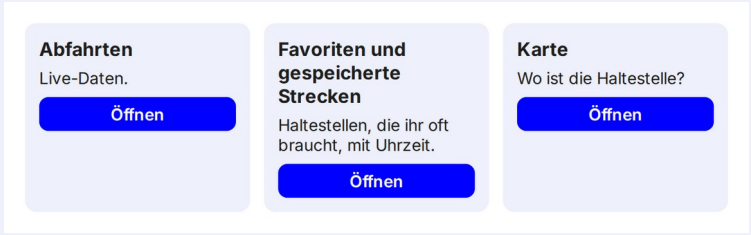
Bei vielen Karten sehen beide gleich aus. Für eine Produktliste meist auto-fill – sonst wird eine einzelne Karte riesig.

© 2026 TAISC – The AI Software Company GmbH

SUBGRID

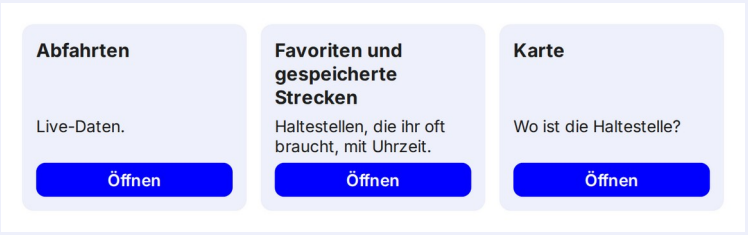
Karten, deren Inhalte sauber fluchten

Ohne subgrid



Jede Karte rechnet für sich – Buttons auf unterschiedlicher Höhe

Mit subgrid



Titel, Text und Button teilen sich die Zeilen des Eltern-Grids

style.css

```
.karten {
  display: grid;
  grid-template-columns:
    repeat(3, 1fr);
}
.karte {
  display: grid;
  grid-row: span 3;
  grid-template-rows: subgrid;
}
```

© 2026 TAISC – The AI Software Company GmbH

FLEX ODER GRID?

Die Entscheidung in fünf Fragen

Frage	Flexbox	Grid
Eine Richtung oder zwei?	eine (Zeile oder Spalte)	zwei (Zeilen und Spalten)
Bestimmt der Inhalt die Größe?	ja – Inhalt zuerst	nein – Raster zuerst
Sollen Elemente fluchten?	nur in ihrer Zeile	über alle Zeilen hinweg
Typische Beispiele	Navbar, Buttons, Tags, Formularzeile	Seitenlayout, Galerie, Dashboard, Karten
Kombinierbar?	ja – Flex in Grid-Zellen ist der Normalfall	ja – Grid für die Seite, Flex im Detail

Faustregel: Grid für das Gerüst, Flex für das, was darin steht.

© 2026 TAISC – The AI Software Company GmbH

DER UNTERSCHIED

Flex oder Grid – am selben Beispiel

Galerie mit Flexbox (flex-wrap + flex: 1 1 160px)



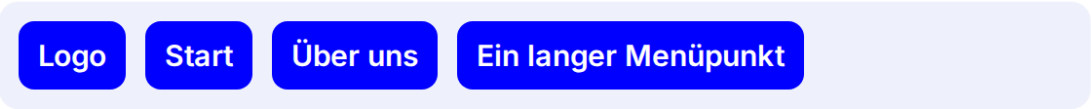
✗ letzte Reihe wird breiter als die oberen

Galerie mit Grid (auto-fill, minmax(160px, 1fr))



✓ alle Karten gleich breit, sauberes Raster

Navigation mit Flexbox



✓ jeder Punkt so breit wie sein Text

Navigation mit Grid (repeat(4, 1fr))



✗ starre Spalten, langer Text wird abgeschnitten

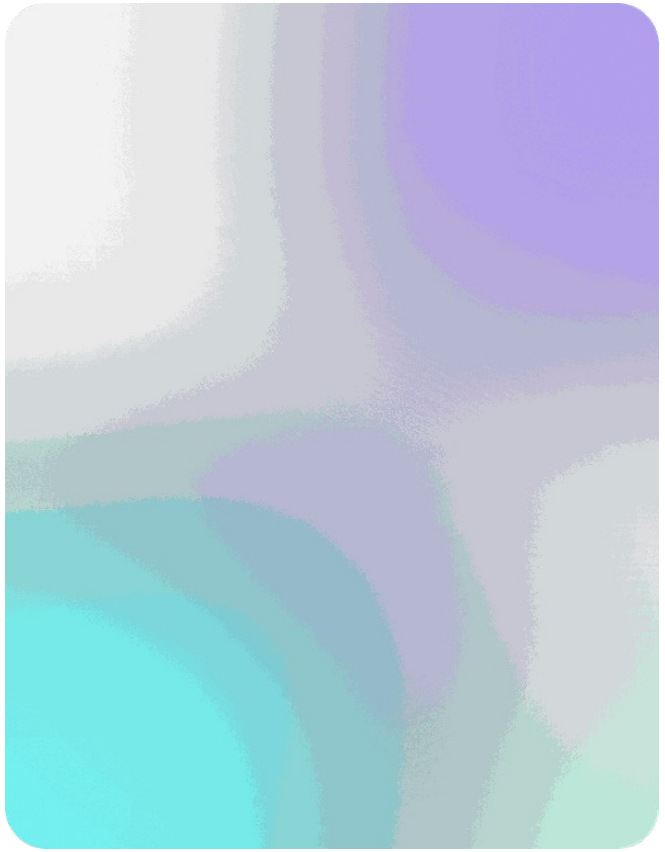
Flexbox denkt vom Inhalt aus, Grid vom Raster aus. Beide sind aktuell – fast jede Seite nutzt beides.

© 2026 TAISC – The AI Software Company GmbH

KAPITEL 2

Responsive

Eine Seite, jedes Gerät.



MOBILE FIRST

Klein anfangen, dann erweitern

style.css

```
/* Handy: eine Spalte */
.karten { display: grid; gap: 1rem; }

/* ab 768 px: zwei Spalten */
@media (min-width: 768px) {
  .karten {
    grid-template-columns: 1fr 1fr;
  }
}
```

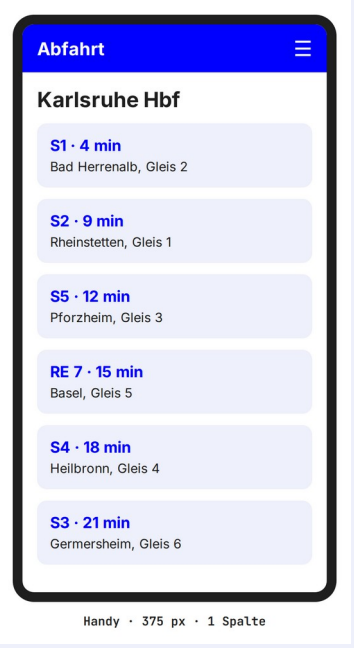
- Mehr als die Hälfte der Aufrufe kommt vom Handy
- Erst die schmale Version, dann mit min-width erweitern
- Ohne viewport-Meta-Tag wirkt nichts davon
- Testen im DevTools Device Mode

© 2026 TAISC – The AI Software Company GmbH

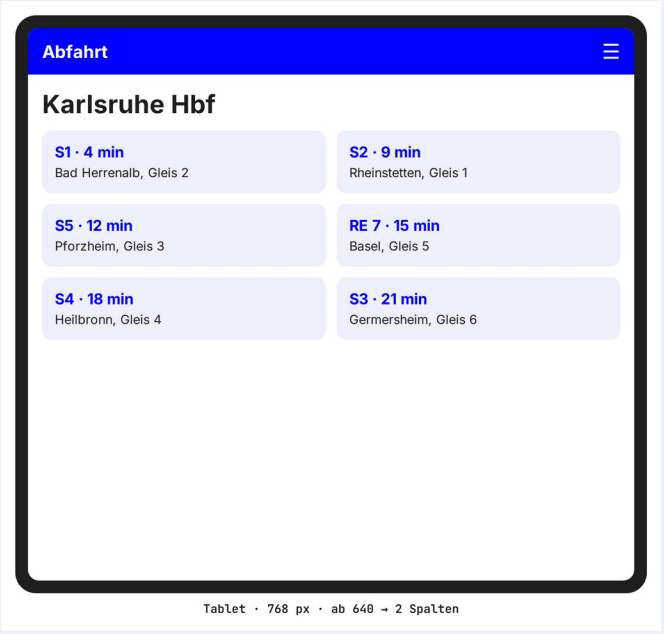
MOBILE FIRST IN BILDERN

Eine Seite, drei Bildschirme

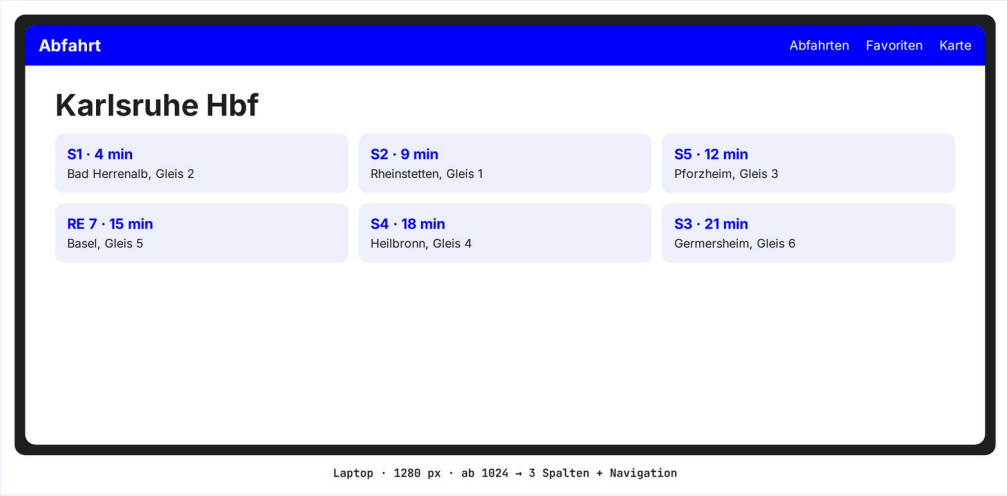
/* Basis */



@media (width >= 640px)



@media (width >= 1024px)



Dieselbe HTML- und CSS-Datei. Der Browser wendet nur die Media Queries an, deren Bedingung gerade zutrifft.

© 2026 TAISC – The AI Software Company GmbH

MEDIA QUERIES VERTIEFT

Mehr als nur Bildschirmbreite

Breite – neue Schreibweise

```
@media (width >= 768px) { … }
@media (640px <= width < 1024px) { … }

/* Hochformat-Tablet */
@media (orientation: portrait) { … }

/* Drucken: Navigation weg */
@media print {
  nav, footer { display: none; }
}
```

Nutzer-Einstellungen

```
/* Maus oder Finger? */
@media (hover: hover) {
  .karte:hover { scale: 1.02; }
}
@media (pointer: coarse) {
  button { min-height: 44px; }
}

@media (prefers-reduced-motion: reduce)
@media (prefers-color-scheme: dark)
@media (prefers-contrast: more)
```

Responsive heißt nicht nur „schmaler Bildschirm“, sondern: Wie und womit benutzt dieser Mensch gerade die Seite?

© 2026 TAISC – The AI Software Company GmbH

IN DER PRAXIS

Woran responsive Seiten scheitern

Breakpoints nach Inhalt

Nicht nach Gerätemodellen. Setzt einen Umbruch dort, wo das Layout hässlich wird – meist um 640, 768 und 1024 px.

Touch-Ziele

Buttons mindestens 44 × 44 px. Was mit der Maus noch trifft, trifft der Daumen nicht mehr.

Tabellen und lange Wörter

overflow-x: auto um die Tabelle, sonst schiebt sie die ganze Seite auf. Lange URLs mit overflow-wrap brechen.

Am echten Gerät testen

Device Mode ist gut, das eigene Handy ist besser. Die Live-URL im Browser des Nachbarn öffnen.

© 2026 TAISC – The AI Software Company GmbH

CONTAINER QUERIES

Eine Karte, die überall passt

style.css

```
.spalte {
  container-type: inline-size;
}

/* Standard: gestapelt */
.karte { display: grid; }

/* wenn MEIN Container
   mind. 420 px breit ist */
@container (width >= 420px) {
  .karte {
    grid-template-columns:
      140px 1fr;
  }
}
```

Gleiche Karte, gleiche Klasse – zwei Container



Media Query fragt: Wie breit ist das Fenster? Container Query fragt: Wie breit ist der Platz, in dem die Komponente steckt?

© 2026 TAISC – The AI Software Company GmbH

FLUIDE GRÖSSEN

Schrift und Abstände ohne Breakpoints

style.css

```
:root {
  --text-xl: clamp(
    1.75rem,         /* min */
    1.2rem + 2.5vw,   /* ideal */
    3.5rem           /* max */
  );
}
h1 {
  font-size: var(--text-xl);
}
```

Dieselbe Überschrift bei drei Breiten



Die Mitte immer aus rem + vw – nur vw würde den Browser-Zoom ignorieren. Werte ausrechnen lassen: utopia.fyi

© 2026 TAISC – The AI Software Company GmbH

RESPONSIVE BILDER

Das richtige Bild für jeden Bildschirm

Die Botschaft: Ein Handy soll nicht dieselbe riesige Datei laden wie ein 4K-Monitor.

1 · srcset: gleiches Motiv, passende Größe

```
<img src="bahn-800.jpg"
  srcset="bahn-400.jpg   400w,
          bahn-800.jpg   800w,
          bahn-1600.jpg 1600w"
  sizes="(width >= 768px) 50vw,
         100vw"
  alt="S1 am Albtalbahnhof"
  width="1600" height="900"
  loading="lazy">
```

2 · Art Direction: anderer Ausschnitt

```
<picture>
  <!-- ab 768 px: Panorama -->
  <source
    media="(width >= 768px)"
    srcset="panorama.jpg">
  <!-- sonst: Hochformat -->
  <img src="hochkant.jpg"
       alt="S1 am Albtalbahnhof">
</picture>
```

3 · Formate: das beste, das der Browser kann

```
<picture>
  <source srcset="bahn.avif"
          type="image/avif">
  <source srcset="bahn.webp"
          type="image/webp">
  <!-- Rückfall für alle -->
  <img src="bahn.jpg"
       alt="S1 am Albtalbahnhof">
</picture>
```

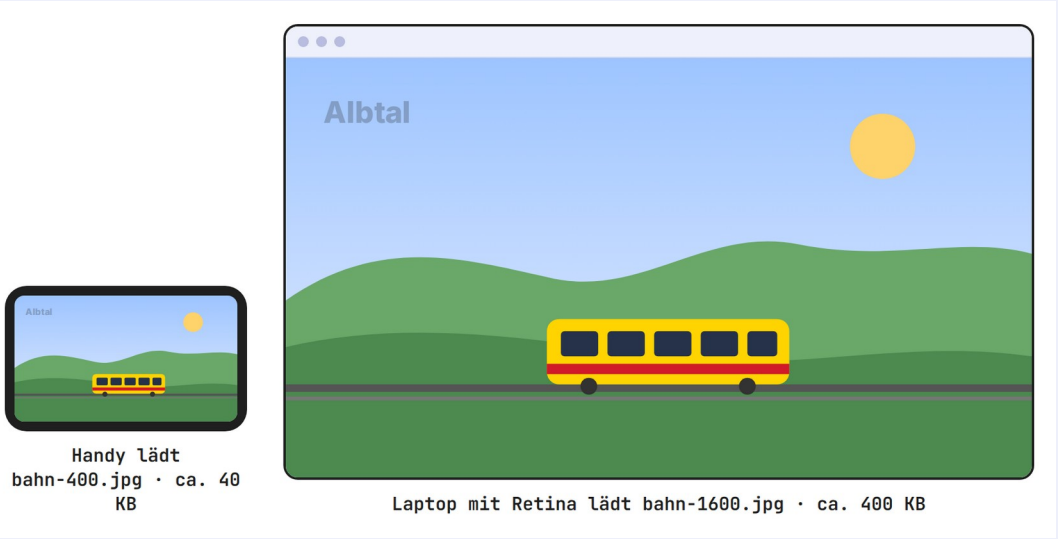
Faustregel: Meistens reicht <img> mit srcset. <picture> nur für einen anderen Ausschnitt oder mehrere Formate.

© 2026 TAISC – The AI Software Company GmbH

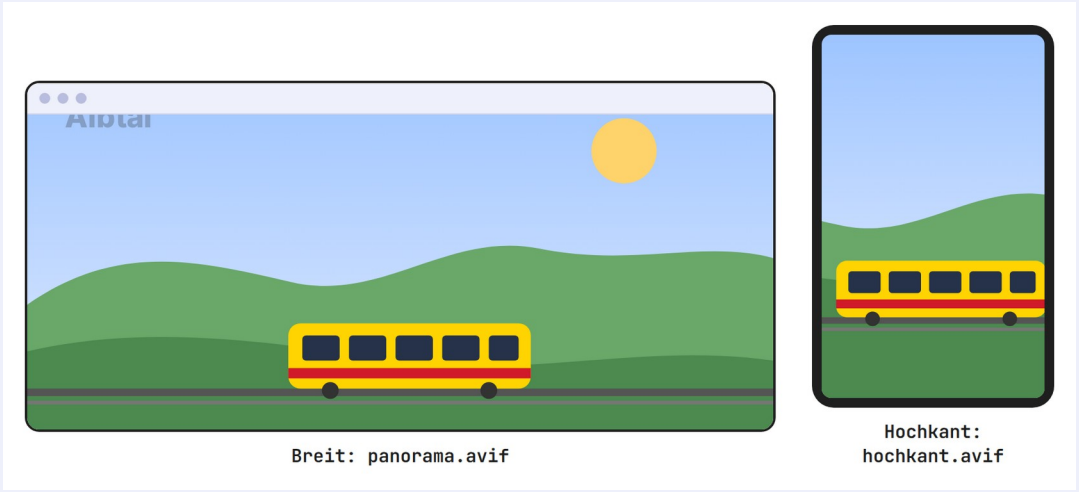
RESPONSIVE BILDER

Was der Browser daraus macht

srcset: gleiches Motiv, passende Größe



<picture>: anderes Motiv je nach Format



Die KB-Angaben sind Beispielwerte. Der Browser wählt nach Breite, Pixeldichte und sizes – ihr liefert nur die Auswahl.

© 2026 TAISC – The AI Software Company GmbH

CSS-VARIABLEN

Farben einmal definieren

style.css

```
:root {
  --blau: #0000ff;
  --radius: 12px;
  --abstand: 1rem;
}

.button {
  background: var(--blau);
  border-radius: var(--radius);
  padding: var(--abstand);
  transition: transform .2s;
}
.button:hover { transform: translateY(-2px); }
```

- **Custom Properties:** Werte an einer Stelle ändern
- Grundlage für Dark Mode und Themes
- **transition + transform:** kleine Animationen ohne JavaScript
- Genau so arbeiten Design-Systeme – auch Tailwind

© 2026 TAISC – The AI Software Company GmbH

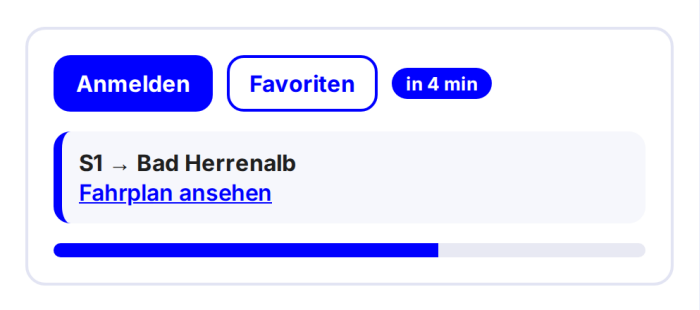
CUSTOM PROPERTIES

Zwei Werte ändern, alles folgt

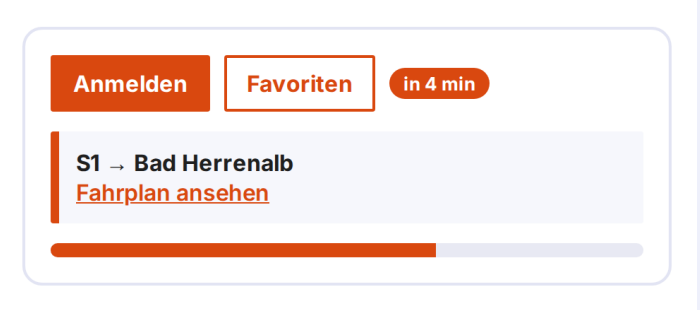
style.css

```
:root {
  --marke: #0000ff;
  --radius: 12px;
}
.button {
  background: var(--marke);
  border-radius: var(--radius);
}
.badge { background: var(--marke); }
.link  { color: var(--marke); }
.karte { border-color: var(--marke); }
```

--marke: #0000ff; --radius: 12px;



--marke: #d9480f; --radius: 2px;



Live mit Farbwähler: Layout-Spielwiese, Reiter „Variablen“.

© 2026 TAISC – The AI Software Company GmbH

DESIGN TOKENS

Wie Design-Systeme Werte organisieren

Basis-Tokens

Was es gibt

- --blau-600: #0000ff
- --grau-100: #f1f5f9
- --abstand-4: 1rem
- Die Palette – noch ohne Bedeutung

Semantische Tokens

Wofür es steht

- --farbe-primär: var(--blau-600)
- --farbe-fläche: var(--grau-100)
- --farbe-fehler: var(--rot-600)
- Dark Mode tauscht nur diese Ebene

Komponenten-Tokens

Wo es gilt

- --button-hintergrund: var(--farbe-primär)
- --karte-radius: var(--radius-l)
- Eine Änderung, alle Buttons folgen

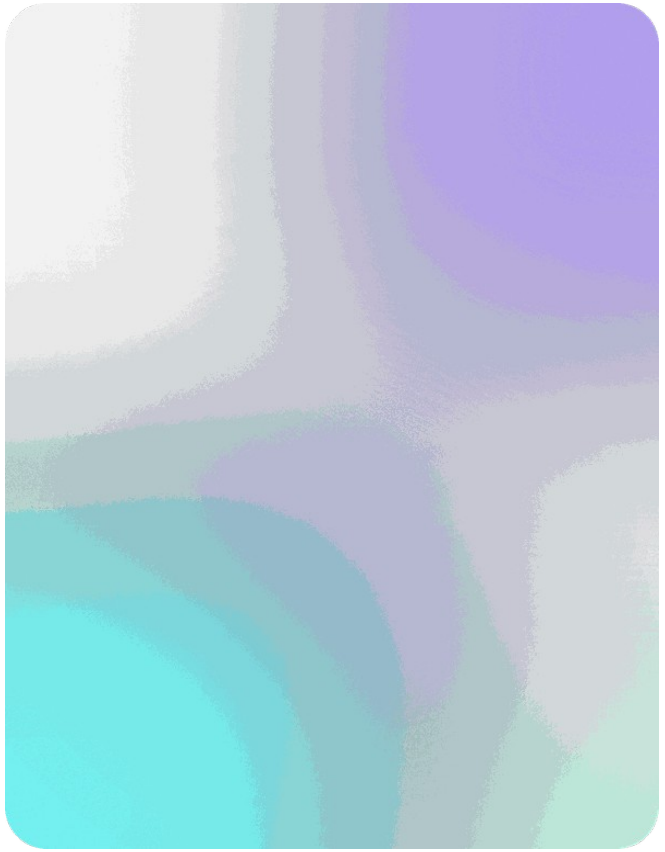
Genau dieses Prinzip steckt in Tailwinds Theme – und in Figma-Variablen, aus denen Designer euch die Werte geben.

© 2026 TAISC – The AI Software Company GmbH

KAPITEL 3

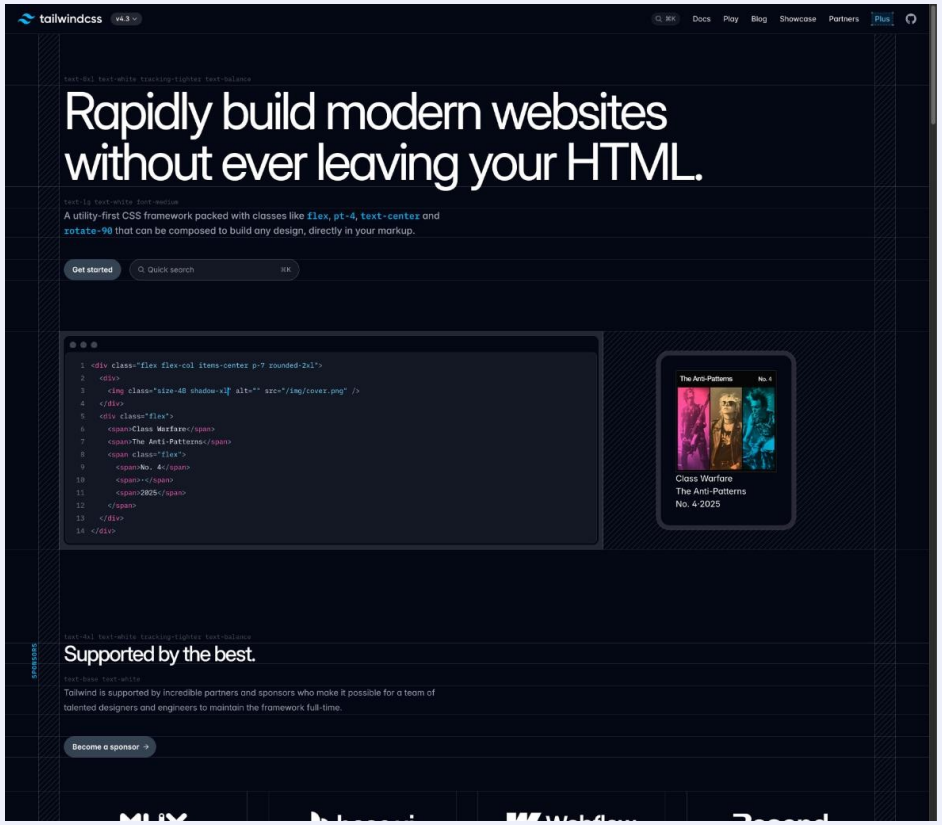
Tailwind CSS

Warum fast jede moderne App so aussieht.



TAILWIND CSS

Was ist Tailwind?



Ein CSS-Framework aus Utility-Klassen

- Erfunden von Adam Wathan, seit 2017 öffentlich
- Gepflegt von Tailwind Labs – einer kleinen Firma aus Kanada
- Open Source (MIT-Lizenz) und kostenlos; Geld verdient die Firma mit Tailwind Plus (fertige Vorlagen)
- Aktuell Version 4.3 (seit Mai 2026)
- Rund 85 Millionen npm-Downloads pro Woche – und null Abhängigkeiten

© 2026 TAISC – The AI Software Company GmbH

GESCHICHTE

Vom Nebenprojekt zum Standard

2017 — 1 — v0.1

Adam Wathan veröffentlicht Tailwind als Nebenprojekt.

2019 — 2 — v1.0

Erste stabile Version, die Community wächst schnell.

2020 — 3 — v2.0

Dark Mode, größere Farbpalette.

2021 — 4 — v3.0

JIT-Compiler: nur benutzte Klassen landen im CSS.

2025 — 5 — v4.0

Neue Engine, Konfiguration direkt im CSS.

2026 — 6 — v4.3

Rekord-Nutzung, aber 75 % der Entwickler entlassen: KI-Tools schicken kaum noch jemanden auf die Doku.

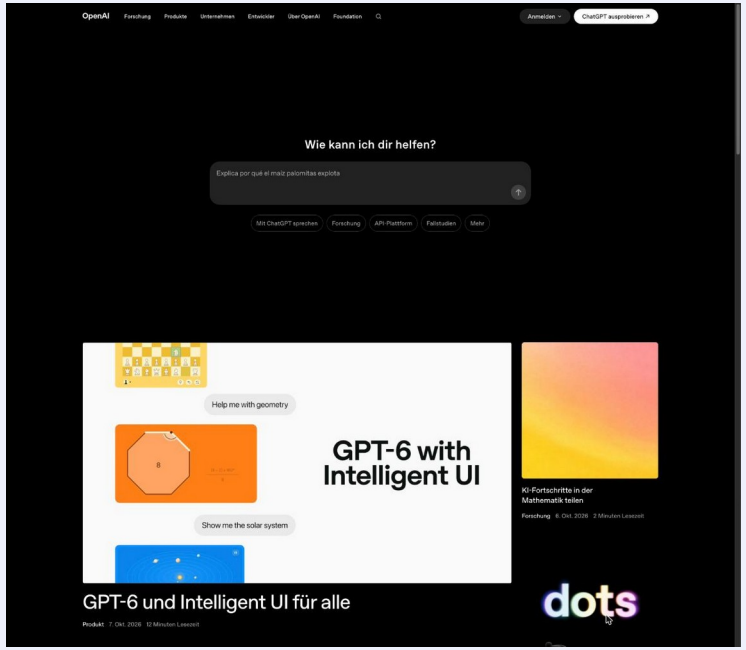
Danach sprangen Sponsoren wie Vercel und Google AI Studio ein. Ein Lehrstück, wie KI Open-Source-Geschäftsmodelle trifft.

© 2026 TAISC – The AI Software Company GmbH

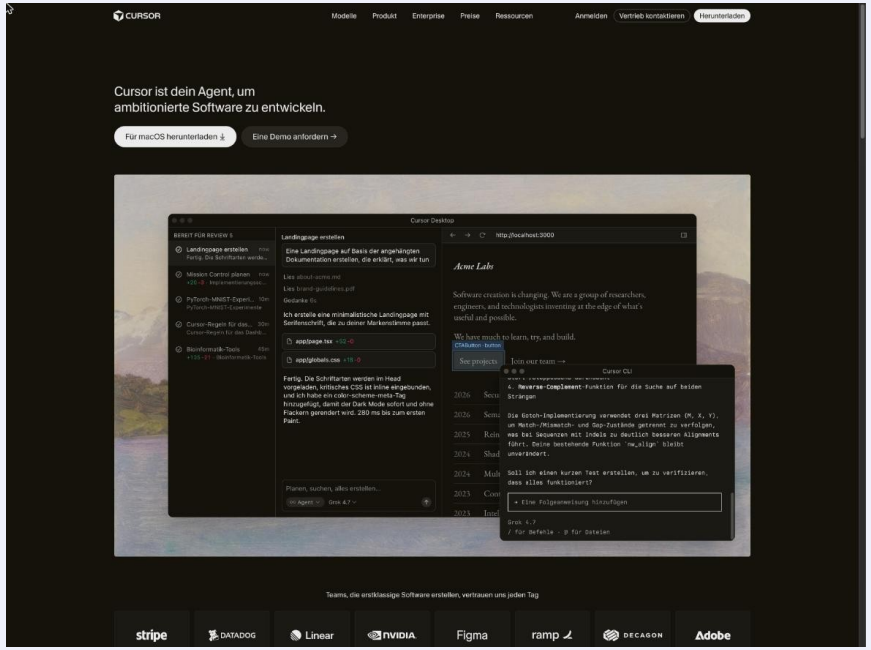
IN DER PRAXIS

Echte Seiten, gebaut mit Tailwind

openai.com



cursor.com



Beide stehen im offiziellen Showcase tailwindcss.com/showcase – neben Spiegel, Shopify und NASA JPL. Erkennbar an den Klassen im Quelltext.

© 2026 TAISC – The AI Software Company GmbH

DAS PROBLEM

Warum großes CSS weh tut

Namen

- Heißt es .card, .karte oder .box-wrapper?
- Jeder im Team benennt anders

Toter Code

- Niemand traut sich, Regeln zu löschen
- Die Datei wächst nur

Nebenwirkungen

- Eine Änderung am Button bricht drei andere Seiten
- Die Kaskade schlägt zurück

Konsistenz

- 14 verschiedene Blautöne
- Abstände 13, 15, 16, 17 px

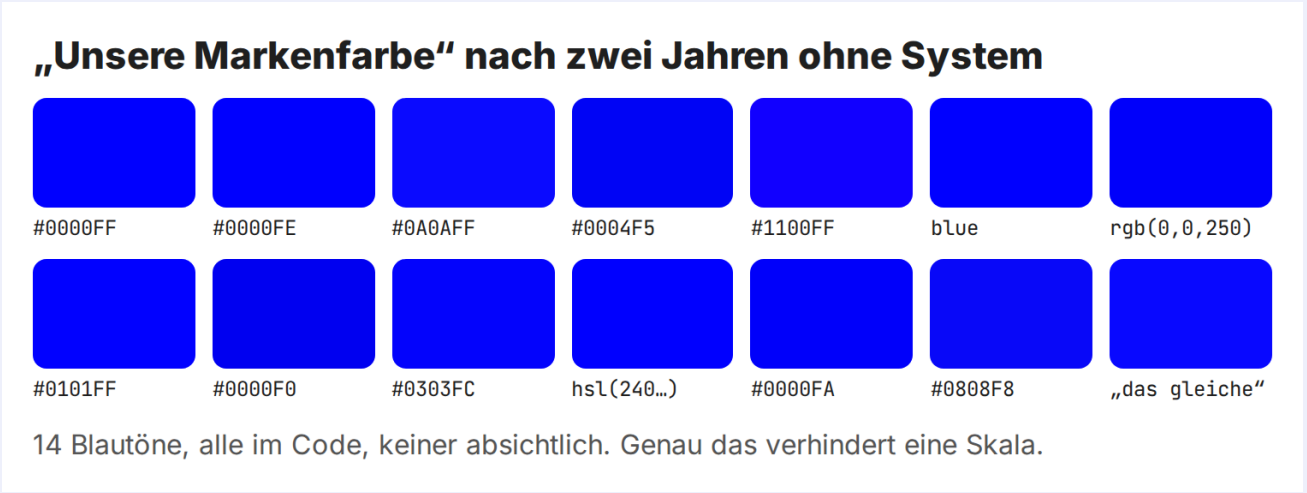
© 2026 TAISC – The AI Software Company GmbH

KENNT IHR DAS?

Wenn CSS ohne System wächst



Der Klassiker: fester Kasten, zu langer Inhalt



Was passiert, wenn jeder seine eigene Farbe tippt

© 2026 TAISC – The AI Software Company GmbH

VERGLEICH

Dieselbe Karte: 33 Zeilen oder 5

style.css (1/2)

```
.karte {
  position: relative;
  padding: 1rem;
  border-radius: .75rem;
  background: #f1f5f9;
}
.karte:hover {
  background: #e2e8f0;
}
@media (width >= 48rem) {
  .karte { padding: 1.5rem; }
}
```

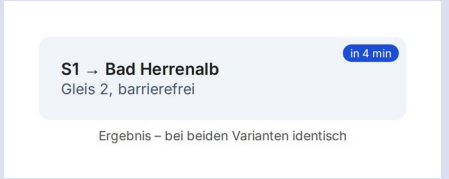
style.css (2/2)

```
.badge {
  position: absolute;
  top: .5rem; right: .5rem;
  padding: .125rem .5rem;
  border-radius: 9999px;
  background: #1d4ed8;
  color: #fff;
  font-size: .75rem;
}
.titel { font-size: 1.125rem;
         font-weight: 600; }
.info  { color: #475569; }
```

index.html

```
<article class="karte">
  <span class="badge">
    in 4 min</span>
  <h2 class="titel">
    S1 → Bad Herrenalb</h2>
  <p class="info">
    Gleis 2, barrierefrei</p>
</article>
```

Ergebnis



Mit Tailwind: alles in index.html

```
<article class="relative rounded-xl bg-slate-100 p-4 hover:bg-slate-200 md:p-6">
  <span class="absolute right-2 top-2 rounded-full bg-blue-700 px-2 py-0.5 text-xs text-white">in 4 min</span>
  <h2 class="text-lg font-semibold">S1 → Bad Herrenalb</h2>
  <p class="text-slate-600">Gleis 2, barrierefrei</p>
</article>
```

© 2026 TAISC – The AI Software Company GmbH

DER VORTEIL

Warum Klassen direkt ins HTML?

Keine Namen erfinden

Ihr müsst nicht überlegen, ob es .karte, .card oder .box-wrapper heißt – und das Team muss es nicht erraten.

Keine Nebenwirkungen

Eine Klasse wirkt nur dort, wo sie steht. Löscht ihr das Element, ist auch sein Stil weg – kein toter Code.

Feste Skala

p-4 ist immer 1rem, blue-700 immer derselbe Ton. Niemand mischt 13 px und 17 px.

Alles an einer Stelle

Struktur und Aussehen in einer Datei – für Menschen und für KI leichter zu lesen und zu ändern.

© 2026 TAISC – The AI Software Company GmbH

ÜBERSETZUNG

Tailwind ist nur CSS mit kurzen Namen

Tailwind-Klasse	Erzeugtes CSS
p-4	padding: 1rem
mt-2	margin-top: 0.5rem
flex justify-between items-center	display: flex; justify-content: space-between; align-items: center
grid grid-cols-3 gap-4	display: grid; grid-template-columns: repeat(3, …); gap: 1rem
text-lg font-bold	font-size: 1.125rem; font-weight: 700
rounded-xl	border-radius: 0.75rem
bg-blue-600 text-white	background-color: …; color: #fff
hover:bg-blue-700	Regel mit :hover
md:grid-cols-2	Regel in @media (min-width: 768px)
sticky top-0 z-10	position: sticky; top: 0; z-index: 10

© 2026 TAISC – The AI Software Company GmbH

DAS SYSTEM DAHINTER

Skalen statt Zufallswerte

Bereich	Skala	Beispiel
Abstände	1 Einheit = 0,25 rem = 4 px	p-4 = 1 rem · gap-6 = 1,5 rem · mt-px = 1 px
Farben	Töne 50 bis 950 je Farbe	bg-slate-50 hell … bg-slate-950 fast schwarz
Schrift	xs, sm, base, lg, xl, 2xl … 9xl	text-base = 1 rem · text-4xl = 2,25 rem
Breakpoints	sm 40 · md 48 · lg 64 · xl 80 · 2xl 96 rem	md: gilt ab 768 px aufwärts
Radien	sm, md, lg, xl, 2xl, 3xl, full	rounded-full = Kreis oder Pille
Schatten	xs, sm, md, lg, xl, 2xl	shadow-lg für schwebende Karten

Die Skala ist der eigentliche Wert von Tailwind: Ein Team kann nicht mehr 13 px und 17 px mischen.

© 2026 TAISC – The AI Software Company GmbH

VARIANTEN

Zustände direkt an die Klasse schreiben

Beispiele

```
<button class="bg-blue-600 hover:bg-blue-700
  focus-visible:outline-2 disabled:opacity-50
  md:px-6 dark:bg-blue-500">

<!-- Elternteil steuert Kind -->
<a class="group">
  <span class="group-hover:underline">…</span>
</a>

<!-- Geschwister steuert Fehlermeldung -->
<input class="peer" type="email" required>
<p class="hidden peer-user-invalid:block">…</p>
```

Was ein Präfix bedeutet

- **hover:, focus-visible:, disabled:** Pseudoklassen
- **md:, lg:** Media Query ab dieser Breite – mobile first
- **dark:** Dark Mode
- **group-*, peer-*:** Reaktion auf Eltern oder Geschwister
- **aria-expanded:, data-[state=open]:** Zustände aus Attributen

© 2026 TAISC – The AI Software Company GmbH

EINORDNUNG

Was Tailwind ist – und was nicht

Tailwind ist

- Ein Satz Utility-Klassen für CSS-Eigenschaften
- Ein Design-System: feste Abstände, Farben, Größen
- Ein Build-Tool, das nur genutzte Klassen ausliefert
- Der Standard bei v0, Lovable, shadcn/ui und Co.

Tailwind ist nicht

- Eine eigene Sprache – dahinter steht CSS
- Magie – ohne Box-Modell und Flexbox versteht ihr es nicht
- Ein Ersatz für semantisches HTML
- Pflicht – kleine Seiten gehen auch ohne

Deshalb erst CSS, dann Tailwind. Umgekehrt versteht man keines von beiden.

© 2026 TAISC – The AI Software Company GmbH

EINBINDEN

Zwei Wege zu Tailwind

Zum Ausprobieren (Play CDN)

```
<script src=
"https://cdn.jsdelivr.net/npm/
@tailwindcss/browser@4"></script>

<h1 class="text-3xl font-bold">
  Hallo
</h1>
```

Für das Projekt (Vite)

```
npm create vite@latest
npm install tailwindcss \
  @tailwindcss/vite

/* style.css */
@import "tailwindcss";
```

Play CDN zum Ausprobieren, Vite für das Projekt – npm erklären wir gleich.

© 2026 TAISC – The AI Software Company GmbH

TAILWIND V4

Konfiguration direkt im CSS

src/style.css

```
@import "tailwindcss";

@theme {
  --color-marke: oklch(45% 0.3 265);
  --color-marke-hell: oklch(92% 0.04 265);
  --font-display: "Inter", sans-serif;
  --breakpoint-3xl: 120rem;
}

/* jetzt verfügbar: */
/* bg-marke, text-marke-hell,
   font-display, 3xl:grid-cols-4 */
```

Was sich mit v4 geändert hat

- Keine tailwind.config.js mehr nötig – das Theme steht im CSS
- Jeder Theme-Wert ist auch eine CSS-Variable: var(--color-marke)
- Klassen werden automatisch im Projekt gefunden
- Viele Tutorials und KI-Antworten zeigen noch v3 – auf die Version achten

© 2026 TAISC – The AI Software Company GmbH

V3 VS. V4

Wozu keine tailwind.config.js mehr?

Früher (v3): zwei Dateien

```
// tailwind.config.js
module.exports = {
  content: ["./src/**/*.html"],
  theme: {
    extend: {
      colors: { marke: "#0000ff" },
    },
  },
};

/* style.css */
@tailwind base;
@tailwind components;
@tailwind utilities;
```

Heute (v4): nur style.css

```
@import "tailwindcss";

@theme {
  --color-marke: #0000ff;
}

/* Fertig: Klassen werden
   automatisch gefunden,
   bg-marke und text-marke
   funktionieren sofort. */
```

Gleiches Ergebnis. v4 liest das Theme aus CSS-Variablen – eine Datei und eine Sprache weniger.

© 2026 TAISC – The AI Software Company GmbH

WENN DIE SKALA NICHT REICHT

Beliebige Werte – sparsam

Arbitrary Values

```
<div class="w-[37rem] bg-[#0000ff]
          grid-cols-[220px_1fr]
          top-[calc(100%-2px)]">

<!-- eigene CSS-Variable -->
<div class="bg-(--farbe-flaeche)">
```

Eigene Utility

```
/* style.css */
@utility karten-raster {
  display: grid;
  grid-template-columns:
    repeat(auto-fill, minmax(16rem, 1fr));
  gap: 1.5rem;
}

<ul class="karten-raster">…</ul>
```

Eckige Klammern sind der Notausgang. Wer sie ständig braucht, sollte die Werte ins Theme aufnehmen.

© 2026 TAISC – The AI Software Company GmbH

LANGE KLASSENLISTEN

Wenn das class-Attribut ausufert

Ohne Komponente

```
<button class="inline-flex
  items-center justify-center
  gap-2 rounded-lg bg-blue-600
  px-4 py-2 text-sm font-semibold
  text-white shadow-sm transition
  hover:bg-blue-700
  focus-visible:outline-2
  focus-visible:outline-offset-2
  focus-visible:outline-blue-600
  disabled:opacity-50">
  Speichern
</button>
<!-- …und das 30 Mal -->
```

Mit Komponente

```
<!-- Button.astro -->
<button class="inline-flex …">
  <slot />
</button>

<!-- überall sonst: -->
<Button>Speichern</Button>
<Button>Abbrechen</Button>
```

Weitere Helfer

- Prettier-Plugin sortiert Klassen immer gleich – Diffs bleiben lesbar
- cn() / tailwind-merge setzt Klassen bedingt zusammen und löst Konflikte wie p-2 gegen p-4
- @apply zieht Klassen in eine CSS-Regel – meist ein Zeichen, dass eine Komponente fehlt

Das ist die Kehrseite von Tailwind: Je mehr Utility-Klassen, desto länger das class-Attribut.

© 2026 TAISC – The AI Software Company GmbH

DER NÄCHSTE SCHRITT

Komponente, Bibliothek – und die KI

Komponente

Ein wiederverwendbares UI-Stück

- HTML-Struktur + Klassen (+ evtl. Verhalten) als ein Baustein
- Button, Karte, Navbar, Modal
- Einmal gebaut, überall per Name benutzt
- In eurem Projekt: mindestens drei

Bibliothek

Fertige Komponenten von anderen

- shadcn/ui: Code wird in euer Projekt kopiert
- daisyUI, Flowbite: fertige Klassen wie btn, card
- MUI: React-Komponenten mit eigenem Design
- Spart Zeit, bringt aber deren Look mit

Warum KI Tailwind liebt

Alles steht an einer Stelle

- Stil steht im HTML – keine zweite Datei als Kontext nötig
- Millionen Beispiele im Trainingsmaterial
- Feste Klassennamen statt erfundener Namen → weniger Fehler
- Nebenwirkung: alles sieht gleich aus

© 2026 TAISC – The AI Software Company GmbH

KOMPONENTEN-BIBLIOTHEKEN

Fertige Bausteine im Vergleich

Bibliothek	Ansatz	Gut, wenn …
shadcn/ui	Code wird in euer Projekt kopiert – gehört dann euch	React + Tailwind, volle Kontrolle
Radix, React Aria	Nur Verhalten und Barrierefreiheit, kein Aussehen	Ihr habt ein eigenes Design
daisyUI	Tailwind-Plugin mit Klassen wie btn, card, modal	Reines HTML, schnell hübsch
Flowbite	Tailwind-Komponenten plus etwas JavaScript	Dashboards, Admin-Oberflächen
Material UI	Googles Material Design als React-Komponenten	Große Apps, fertiges Look-and-feel
Bootstrap	Der Klassiker – eigenes CSS, eigene Klassen	Bestehende Projekte, schnelle Prototypen

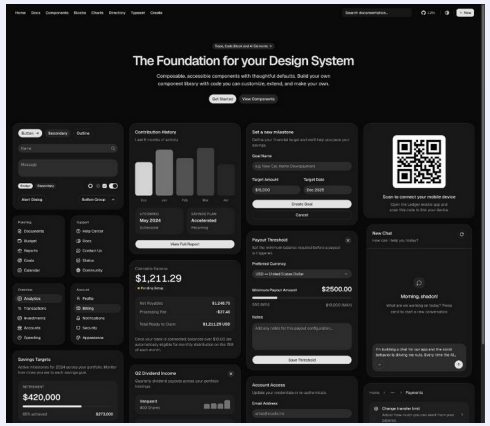
Für euer Projekt: daisyUI für reines HTML oder einzelne Muster bei shadcn/ui abschauen – verstehen statt kopieren.

© 2026 TAISC – The AI Software Company GmbH

KOMPONENTEN-BIBLIOTHEKEN

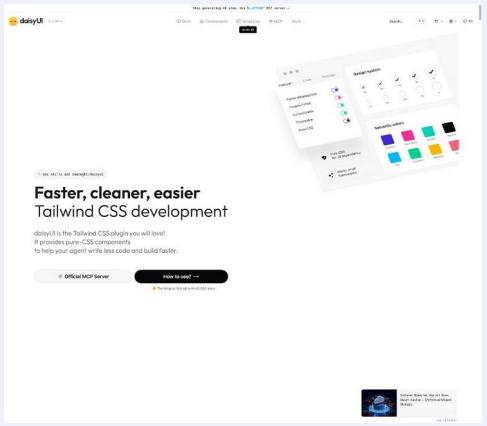
So sehen die Bibliotheken aus

shadcn/ui



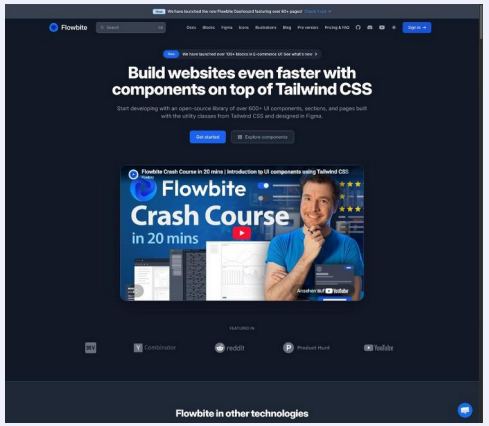
github.com/shadcn-ui/ui

daisyUI



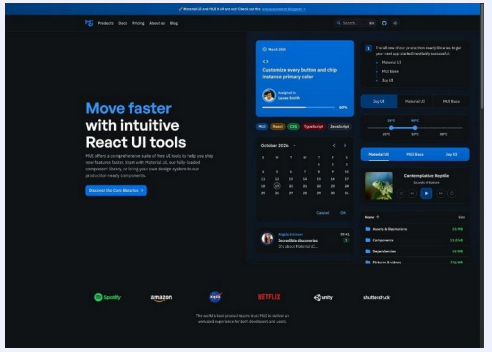
github.com/saadeghi/daisyui

Flowbite



github.com/themesberg/flowbite

MUI (Material UI)



github.com/mui/material-ui

Screenshots der Startseiten, Oktober 2026. Auf jeder Seite gibt es eine Komponenten-Übersicht zum Durchklicken.

© 2026 TAISC – The AI Software Company GmbH

DER KI-LOOK

Warum so viele KI-Apps gleich aussehen

Die Ursache

- v0, Lovable, Bolt, Claude: fast immer Tailwind + shadcn/ui
- Standardfarben slate und indigo, Radius lg, Inter
- Lila Verlauf im Hero, drei Feature-Karten darunter
- Das Modell nimmt den häufigsten Fall

Der Ausweg

- Eigene Tokens im @theme: Farben, Schrift, Radien
- Eine Referenz mitgeben: Screenshot, Website, Figma
- Typografie bewusst wählen – sie prägt den Eindruck am stärksten
- Weißraum und Hierarchie selbst festlegen

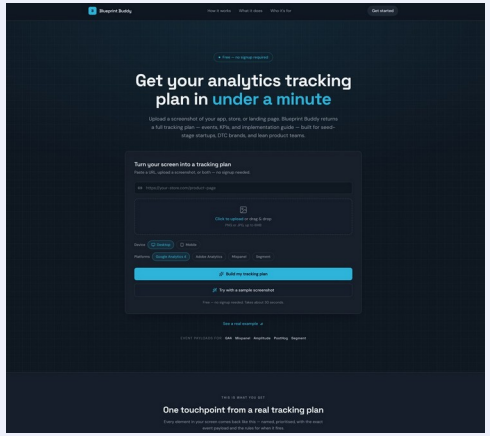
Für euch

- Die KI liefert den ersten Entwurf in Minuten
- Euer Wert: erkennen, was generisch ist
- Und es mit CSS-Wissen gezielt ändern

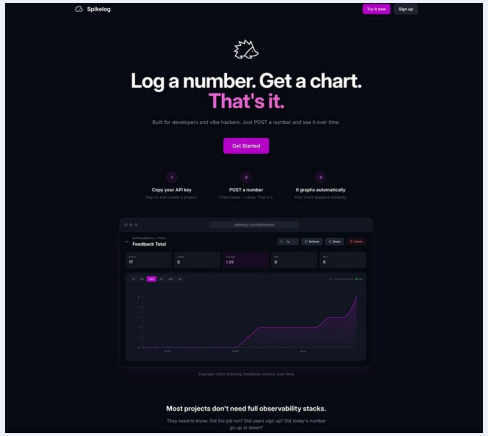
© 2026 TAISC – The AI Software Company GmbH

DER KI-LOOK

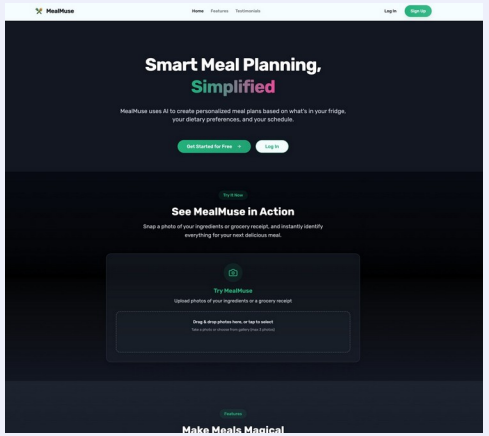
Vier echte Startups, ein Look



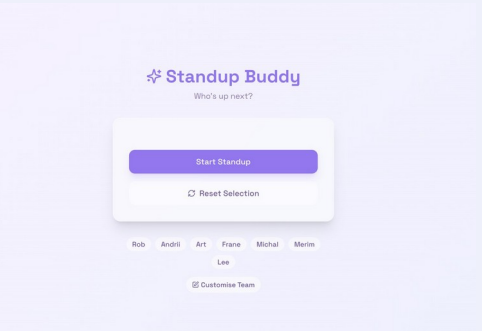
Blueprint Buddy



Spikelog



MealMuse



Standup Buddy

Zentrierter Hero, Verlauf oder Neon-Farbe im Titel, „Free – no signup“-Pill, dunkler Hintergrund, Upload-Karte. Erkennt ihr das Muster?

© 2026 TAISC – The AI Software Company GmbH

FÜR EUER PROJEKT

Bitte kein AI-Slop

KI darf beim Bauen helfen. Wie es aussieht, entscheidet ihr: www.awwwards.com

Eigene Farben

Eine eigene Palette im @theme – nicht die Standardwerte slate, indigo und den lila Verlauf.

Bewusste Schrift

Eine Schrift, die zum Thema passt. Nicht automatisch Inter oder Geist.

Echte Inhalte

Keine „Supercharge your workflow“-Texte, keine erfundenen Testimonials, keine Platzhalter-Features.

Referenz statt Zufall

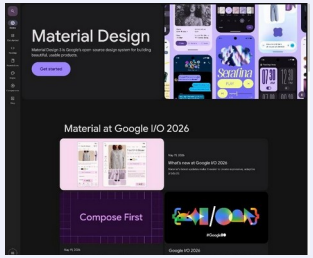
Gebt der KI ein Vorbild mit (Screenshot, Moodboard) und korrigiert gezielt mit eurem CSS-Wissen.

© 2026 TAISC – The AI Software Company GmbH

DESIGN-SYSTEME

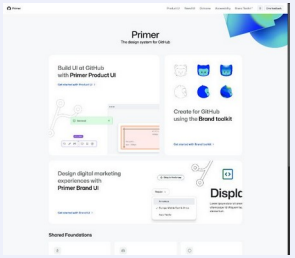
Wie es die Großen machen

Material Design · Google



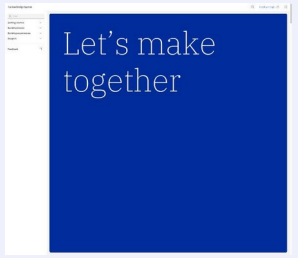
m3.material.io

Primer · GitHub



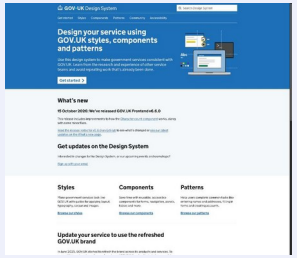
primer.style

Carbon · IBM



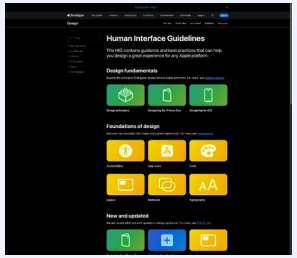
carbondesignsystem.com

GOV.UK Design System



design-system.service.gov.uk

Apple Human Interface Guidelines



developer.apple.com/design

Was ihr dort findet

- Farben, Abstände, Schrift als Tokens
- Komponenten mit Regeln: wann welcher Button
- Texte: wie man Fehlermeldungen schreibt
- Alles öffentlich – zum Lernen, nicht zum Kopieren

© 2026 TAISC – The AI Software Company GmbH

EINORDNUNG

Tailwind und die Alternativen

Ansatz	Stärke	Schwäche
Reines CSS	Kein Werkzeug, volle Kontrolle, Standard	Ordnung muss man selbst halten
Sass	Variablen, Mixins – war vor nativen CSS-Features unverzichtbar	Vieles kann CSS heute selbst
CSS Modules	Klassen gelten nur in einer Komponente	Braucht einen Build
Bootstrap	Schnell, bekannt, viele Vorlagen	Sieht schnell nach Bootstrap aus
CSS-in-JS	Styles direkt im JavaScript (styled-components)	Laufzeitkosten, auf dem Rückzug
Tailwind	Skala, kein Namensstreit, ideal für KI und Komponenten	Langes HTML, eigene Lernkurve

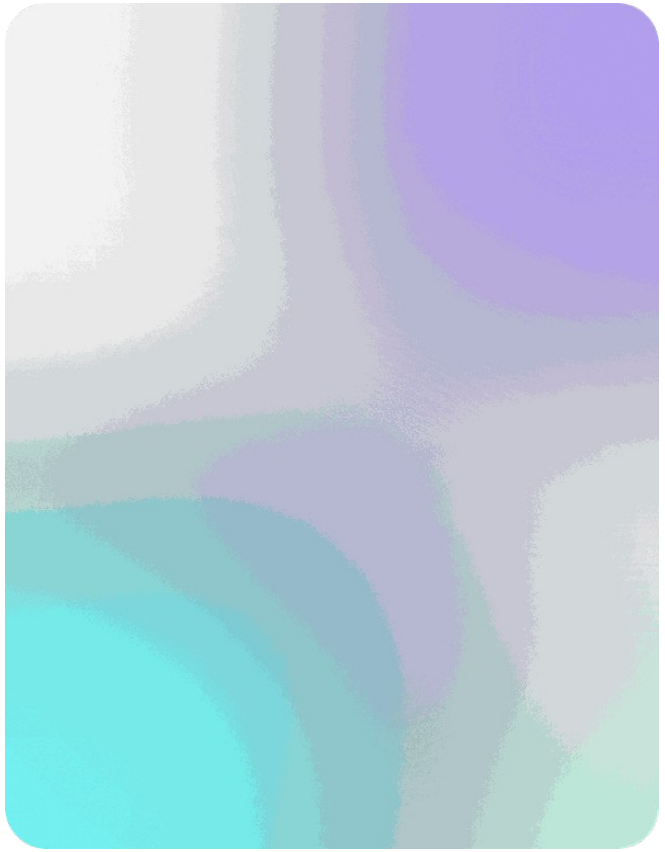
Es gibt kein Richtig. In Jobs begegnet ihr allen sechs – wer CSS versteht, liest jede davon.

© 2026 TAISC – The AI Software Company GmbH

KAPITEL 4

npm & Toolchain

Woher der Code kommt, den ihr nicht selbst schreibt.



NPM

Was ist npm – und was ist Node.js?

Node.js

JavaScript ohne Browser

- Läuft auf eurem Rechner und auf Servern
- Werkzeuge wie Vite und Tailwind sind selbst JavaScript-Programme
- Bringt npm gleich mit

npm – das Programm

Node Package Manager

- npm install lädt Pakete und deren Abhängigkeiten
- Schreibt mit, was installiert wurde: package.json
- Startet eure Befehle: npm run dev

npm – die Registry

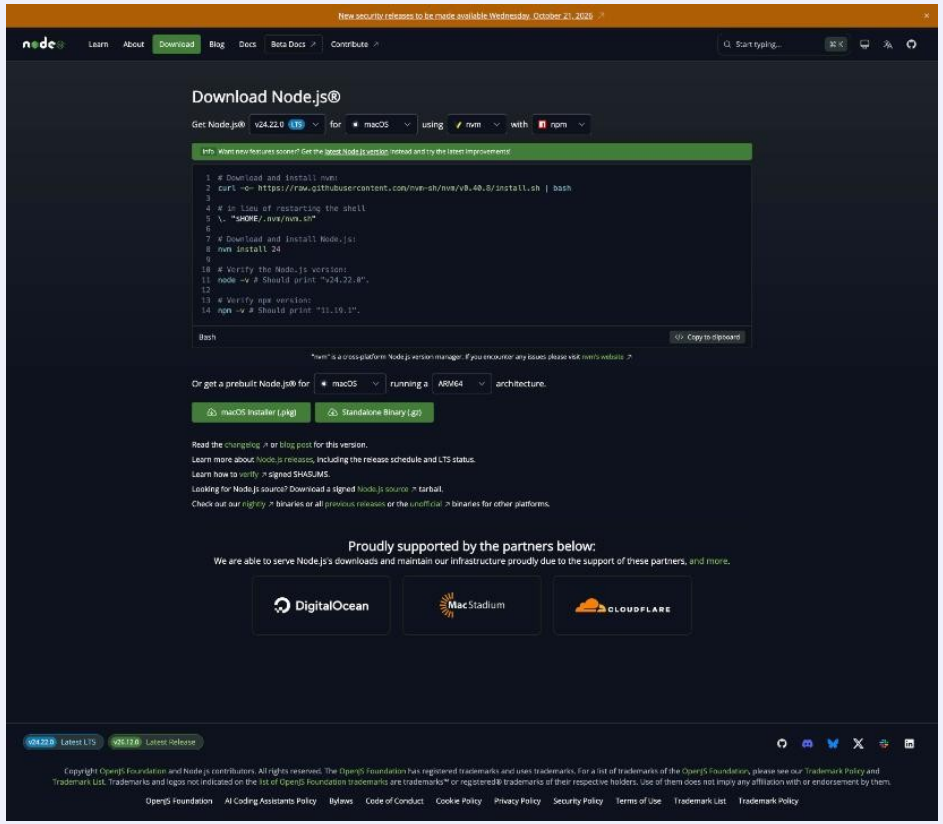
npmjs.com, das Paket-Archiv

- Millionen Pakete, jeder darf veröffentlichen
- tailwindcss: ca. 85 Mio. Downloads pro Woche
- Alternativen mit derselben Registry: pnpm, yarn, bun

© 2026 TAISC – The AI Software Company GmbH

INSTALLIEREN

Node.js und npm installieren



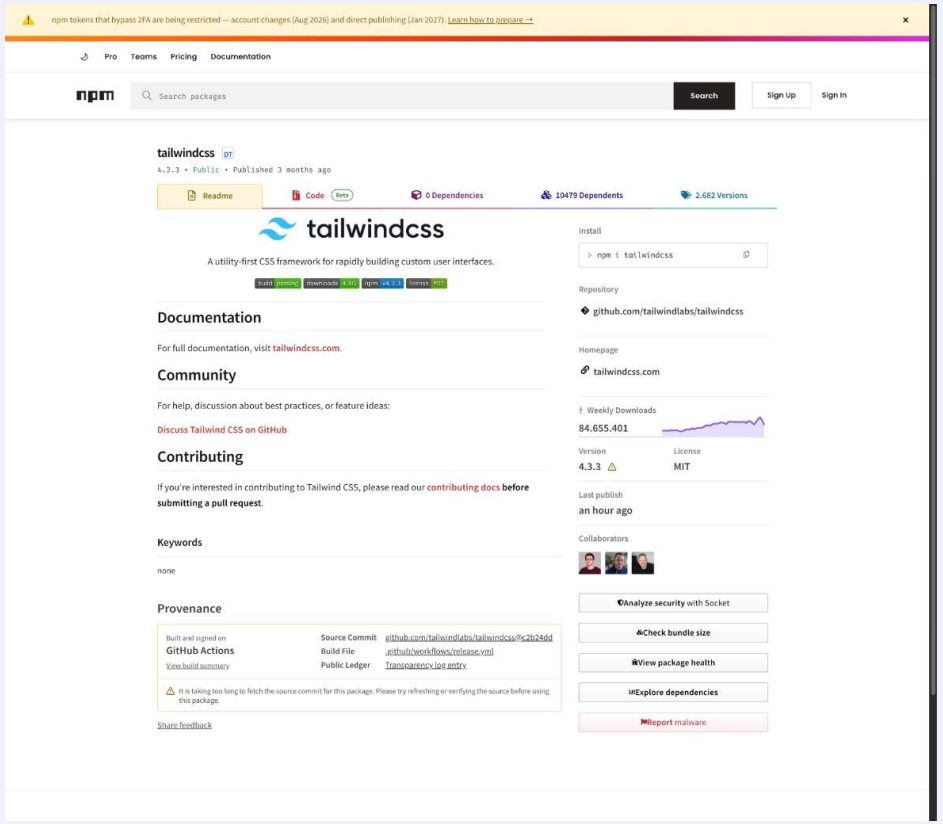
Einmal pro Rechner, fünf Minuten

- nodejs.org → Download → die LTS-Version nehmen (aktuell 24)
- Installer ausführen, Terminal neu öffnen
- Prüfen: node -v → v24.x und npm -v → 11.x
- Profi-Variante: nvm (Node Version Manager) – mehrere Versionen parallel
- Danach im Projektordner: npm install

© 2026 TAISC – The AI Software Company GmbH

PAKETE PRÜFEN

Ein Paket auf npmjs.com lesen



Worauf ihr schaut, bevor ihr installiert

- Weekly Downloads: Wird es wirklich benutzt?
- Version und Last publish: Wird es gepflegt?
- Dependencies: Wie viel kommt mit? (Tailwind: 0)
- Repository: Den Code auf GitHub ansehen
- License: Dürft ihr es benutzen? (MIT = ja)

© 2026 TAISC – The AI Software Company GmbH

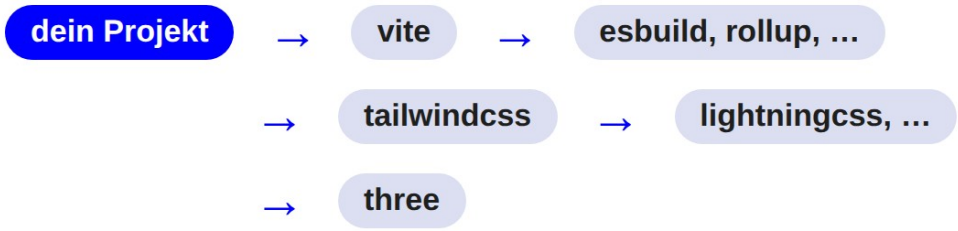
ABHÄNGIGKEITEN

Ein Paket kommt selten allein

PACKAGE.JSON

```
{
  "name": "abfahrtsmonitor",
  "scripts": {
    "dev": "vite"
  },
  "dependencies": {
    "three": "^0.180.0"
  },
  "devDependencies": {
    "vite": "^7.1.0",
    "tailwindcss": "^4.1.0"
  }
}
```

NPM INSTALL



node_modules/
3 Einträge in package.json → oft mehrere Hundert Ordner. Nie committen (.gitignore), jederzeit neu installierbar.

© 2026 TAISC – The AI Software Company GmbH

PACKAGE.JSON

Der Bauplan eures Projekts

package.json

```
{
 "name": "abfahrtsmonitor",
 "type": "module",
 "scripts": {
  "dev": "vite",
  "build": "vite build",
  "preview": "vite preview"
 },
 "dependencies": {
  "three": "^0.180.0"
 },
 "devDependencies": {
  "vite": "^6.0.0"
 }
}
```

- **scripts:** eigene Befehle, gestartet mit npm run dev
- **dependencies:** braucht die fertige App
- **devDependencies:** nur beim Entwickeln nötig
- **package-lock.json** friert exakte Versionen ein – gehört ins Repo
- **node_modules** gehört in .gitignore

© 2026 TAISC – The AI Software Company GmbH

BEFEHLE

npm in sechs Befehlen

Terminal

```
npm create vite@latest projekt
cd projekt
npm install          # alles aus package.json
npm install three    # Paket hinzufügen
npm run dev          # Dev-Server starten

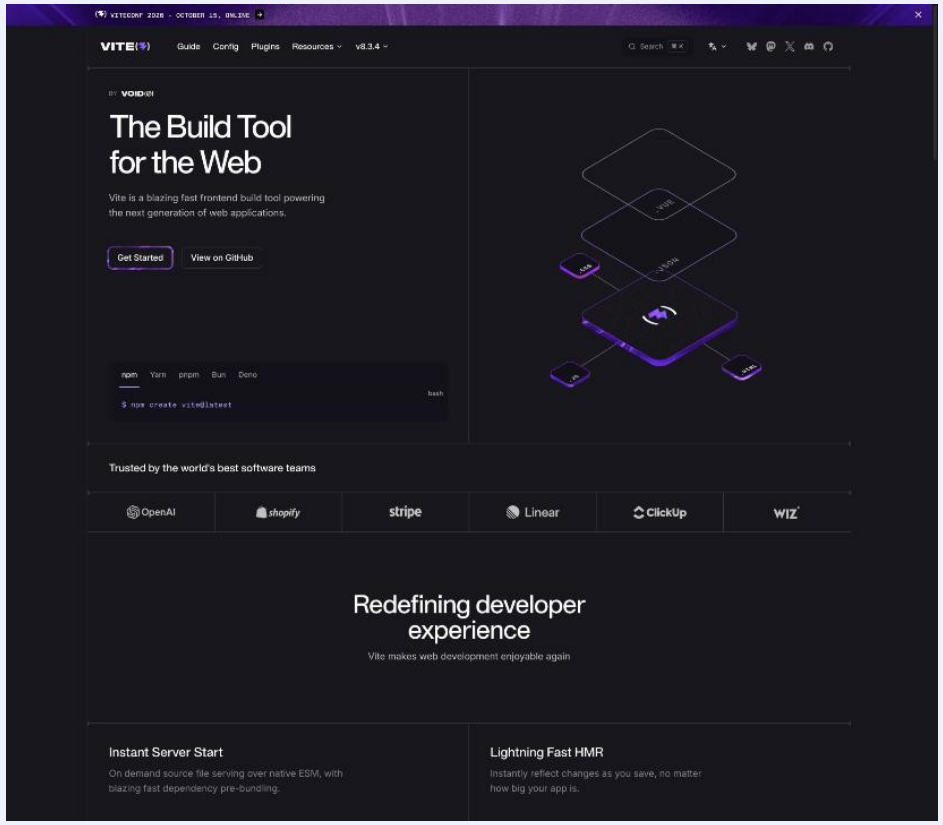
npx serve .          # Paket einmalig ausführen
```

- npm ist Paketmanager und Registry zugleich – mit Millionen Paketen
- npm install liest package.json und füllt node_modules
- npx führt ein Paket aus, ohne es zu installieren
- Alternativen mit gleicher Idee: pnpm, yarn, bun

© 2026 TAISC – The AI Software Company GmbH

VITE

Was ist Vite – und wozu brauchen wir es?



Das Werkzeug zwischen eurem Code und dem Browser

- Dev-Server: npm run dev startet localhost:5173, jede Änderung erscheint sofort im Browser
- Build: npm run build bündelt und verkleinert alles nach dist/
- Plugins: @tailwindcss/vite macht aus euren Klassen echtes CSS
- Ohne Build-Tool versteht der Browser @import "tailwindcss" nicht
- Aktuell Vite 8, rund 65 Mio. Downloads pro Woche

© 2026 TAISC – The AI Software Company GmbH

PROJEKTSTRUKTUR

So sieht ein Vite-Projekt aus

Ordner

```
projekt/
├── index.html       ← Einstieg
├── src/
│   ├── main.js      ← importiert CSS + JS
│   └── style.css    ← @import "tailwindcss"
├── public/          ← Bilder, favicon
│                      (unverändert kopiert)
├── dist/            ← Ergebnis von build
│                      (nicht committen)
├── node_modules/    ← Pakete (nie committen)
├── package.json
├── package-lock.json
└── vite.config.js
```

Was wohin gehört

- **src/** ist euer Code, **dist/** ist das, was ausgeliefert wird
- **public/** für Dateien mit festem Namen
- In **.gitignore**: node_modules und dist
- Vite startet in Millisekunden, weil der Browser die Module im Dev-Modus direkt lädt

© 2026 TAISC – The AI Software Company GmbH

LIVE-DEMO

Von null zu Tailwind in fünf Minuten

1 Projekt anlegen

1 Min.

npm create vite@latest demo → Vanilla → JavaScript. cd demo

2 Tailwind holen

1 Min.

npm install tailwindcss @tailwindcss/vite, Plugin in vite.config.js eintragen

3 Einbinden

1 Min.

@import "tailwindcss"; in src/style.css, npm run dev

4 Bauen & prüfen

2 Min.

Klassen ins HTML, speichern, Browser aktualisiert sich. npm run build → dist/ ansehen.

Danach: dist/ öffnen und zeigen, wie klein das CSS ist – nur die benutzten Klassen sind drin.

© 2026 TAISC – The AI Software Company GmbH

VERSIONEN

Was ^1.4.2 bedeutet

Semantic Versioning: MAJOR.MINOR.PATCH – drei Zahlen mit klarer Bedeutung.

Angabe	Erlaubt	Bedeutung
1.4.2	genau diese	Nichts ändert sich – am sichersten, veraltet am schnellsten
~1.4.2	1.4.x	Nur Patches: Fehlerkorrekturen
^1.4.2	1.x.x	Patches und neue Funktionen, keine Brüche
*** oder latest**	alles	Finger weg in echten Projekten
MAJOR	1.0.0 → 2.0.0	Etwas funktioniert nicht mehr wie vorher
MINOR	1.4.0 → 1.5.0	Neue Funktion, alles Alte läuft weiter
PATCH	1.4.2 → 1.4.3	Fehlerkorrektur

Wenn ein Update etwas zerbricht: erst die MAJOR-Zahl prüfen, dann das Changelog lesen.

© 2026 TAISC – The AI Software Company GmbH

SUPPLY-CHAIN-ATTACKS

Wenn ein Paket zum Angriff wird

Euer Projekt vertraut hunderten Fremden. Das ist kein theoretisches Risiko.

2016 · left-pad

Ein Entwickler zieht ein Elf-Zeilen-Paket zurück – tausende Builds weltweit brechen, darunter große Projekte.

2018 · event-stream

Ein neuer „Helfer“ übernimmt ein beliebtes Paket und baut Code ein, der Krypto-Wallets ausspäht.

2025 · chalk & debug

Per Phishing gekapertes Maintainer-Konto: Schadcode in Paketen mit Milliarden Downloads pro Woche.

Was ihr tun könnt

package-lock.json committen, npm audit laufen lassen, keine Pakete für Kleinkram, Namen genau prüfen.

© 2026 TAISC – The AI Software Company GmbH

BUILD

Warum überhaupt ein Build?

Ohne Build

```
<script src="app.js"></script>
<script src="chart.js"></script>
<link rel="stylesheet"
      href="style.css">
```

→ jede Datei ein Request
→ kein import aus npm
→ kein Tailwind-Kompilieren
→ nichts wird optimiert

Mit Vite

npm run dev
→ Dev-Server, Änderungen sofort sichtbar

npm run build
→ alles gebündelt, minimiert, versioniert
→ landet in dist/
→ dist/ kommt auf den Server

Der Build macht aus eurem Projektordner die paar Dateien, die der Browser wirklich bekommt.

© 2026 TAISC – The AI Software Company GmbH

GRENZEN DER KI

Pakete, Versionen, Halluzinationen

Die KI kennt den Stand ihres Trainings – nicht den von heute.

Alte Versionen

- Tailwind-v3-Setup statt v4, alte Next.js-Befehle
- Selbst wenn ihr die Version nennt, nimmt sie oft die bekannte
- Aktuelle Doku oder Beispiele mitgeben

Erfundene Pakete

- Die KI nennt Pakete, die es nicht gibt
- Angreifer registrieren genau solche Namen
- npm install ohne Prüfen = fremder Code auf eurem Rechner

So prüft ihr

- Auf npmjs.com: existiert es, wer pflegt es?
- Downloads pro Woche, letzte Version
- Braucht ihr das Paket überhaupt?

© 2026 TAISC – The AI Software Company GmbH

WAS IHR MITNEHMEN SOLLTET

Der Umgang mit fremdem Code

Jede Abhängigkeit ist eine Entscheidung

Ein npm install zieht oft dutzende weitere Pakete mit. Für drei Zeilen Code lohnt sich das selten.

Prüfen vor Installieren

Wöchentliche Downloads, letzter Commit, offene Issues, Lizenz. Fünf Minuten auf npmjs.com und GitHub.

Updates sind Arbeit

npm outdated zeigt, was alt ist. Bei MAJOR-Sprüngen vorher das Changelog lesen.

Für euer Projekt

Tailwind und Vite reichen. Three.js, wenn ihr den Bonus wollt. Alles andere begründen.

© 2026 TAISC – The AI Software Company GmbH

NACH DER ZWEITEN INPUT-RUNDE

Offene Werkstatt

Übungsblatt 4 liegt im GitHub-Repo. Ich komme rum – fragt alles.

Danke.

Fragen?

Nächster Termin: 27.10. · Client, Server & HTTP



Joshua Heller

Dozent Web-Engineering 1 · Gründer TAISC

M: dhbw@theaisoftwarecompany.com
L: linkedin.com/in/heller-joshua
W: theaisoftwarecompany.com/karriere